

Bachelor Thesis (173312)
Angewandte Informatik (SPO1a)

Einsatz von Künstlicher Intelligenz zur Testfallgenerierung

Marvin Müller*

18. Februar 2025

Eingereicht bei Prof. Dr. rer. nat. Nicole Ondrusch

*212117, mmueller3@stud.hs-heilbronn.de

Inhaltsverzeichnis

Abkürzungsverzeichnis	IV
Abbildungsverzeichnis	V
Gender-Hinweis	VI
Abstract	VII
Zusammenfassung	VIII
1 Einleitung	1
1.1 Motivation	1
1.2 Forschungsfrage	2
1.3 Ziel der Arbeit	3
1.4 Vorgehensweise	3
2 Grundlagen und Hintergrund	4
2.1 KI	4
2.1.1 Allgemein	4
2.1.2 Bereiche der KI	6
2.2 Large Language Model (LLM)	8
2.2.1 Entwicklungen im Bereich LLM	9
2.3 Prompting	9
2.3.1 Prompt Engineering und Prompt-Techniken	11
2.4 Retrieval Augmented Generation (RAG)	11
2.4.1 Einsatzmöglichkeiten von RAG	13
2.5 Testfall	13
2.5.1 Aufbau eines Testfalls	14
2.6 Anforderungsspezifikation	15
2.6.1 Bedeutung der Anforderungsspezifikation für die Testfallgenerierung	16
3 Related Work und aktuelle Forschung	17
4 Methodik	21
4.1 Verwendetes LLM	21
4.2 Testdaten	23
4.3 Prompt und verwendete Methoden	24
4.4 Auswertungsmetriken	26
4.4.1 Eigenschaften guter Testfälle	26
4.4.2 Ausschlusskriterien	28
4.4.3 Qualitätsmetriken	31
4.4.4 Qualitätskriterien	33
5 Ergebnisse	35
5.1 Vorliegende Ergebnisse	35
5.2 Auswertung der Ergebnisse	35

5.3 Diskussion	35
6 Fazit und Ausblick	37
Literatur	38
Eidesstattliche Erklärung	42
Anhang	43

Abkürzungsverzeichnis

AI (KI): Artificial Intelligence (Deutsch: Künstliche Intelligenz)

GUI: Graphical User Interface (Deutsch: Grafische Benutzeroberfläche)

IEC: International Electrotechnical Commission

IEEE: Institute of Electrical and Electronics Engineers

ISO: International Organization for Standardization

ISTQB: International Software Testing Qualifications Board

LLM: Large Language Model (Deutsch: Großes Sprachmodell)

NLP: Natural Language Processing (Deutsch: Verarbeitung natürlicher Sprache)

RAG: Retrieval Augmented Generation

Abbildungsverzeichnis

2.1	Die Grafik von Hecker et al. (2018) zeigt die Interaktion eines KI-Systems mit seiner Umwelt	5
2.2	Die Grafik von Mocko und Bitton-Bailey (2021) zeigt die Beziehung zwischen KI, maschinellem Lernen, neuronalen Netzen und Deep Learning . .	6
2.3	Die Grafik von Dymatrix (2018) zeigt ein einfaches Neuronales Netz und ein Deep Learning Netz	7
2.4	Die Grafik von Hodiak (2024) zeigt LLM und NLP und verschiedene Anwendungsfälle der Technologien	8
2.5	Ein von Müller (2025) dem Autor dieser Arbeit erstellter Screenshot eines Beispiels für einen Prompt in ChatGPT-4 von OpenAI (2025) in der Benutzeroberfläche	10
2.6	Die Grafik von Honroth et al. (2024) zeigt, wie Retrieval Augmented Generation (RAG) im Detail funktioniert	12
2.7	Die Grafik von Enderli (2019) zeigt einen Testfall im SAP Solution Manager	14
2.8	Die Grafik von Jama Software (2024) zeigt die Eigenschaften von funktionalen vs nichtfunktionalen Requirements	15
4.1	Ein von Müller (2025) dem Autor dieser Arbeit erstellter Screenshot, welcher die Erinnerungsfunktion von ChatGPT-4 von OpenAI (2025) in den Einstellungen zeigt	22
4.2	Eine von Pavankumar21github (2022) erstellte Anforderungsspezifikation, welcher die Anforderungen für die Erstellung eines Accounts in OpenCart (2025) vorgibt	23
4.3	Ein von Pavankumar21github (2022) erstellter Testfall, welcher die Erstellung eines Accounts in OpenCart (2025) anhand der oben gezeigten Anforderungsspezifikation testet	24
4.4	Ein von Müller (2025), dem Autor dieser Arbeit erstellter Screenshot, welcher den Prompt an ChatGPT-4 von OpenAI (2025) zeigt	25
4.5	Die Grafik von Tran et al. (2025) zeigt, wie die einzelnen Qualitätskriterien von den Teilnehmern ihrer Befragung in ihrer Wichtigkeit erachtet werden	27
4.6	Ein von ChatGPT-4 von OpenAI (2025) erstellter Testfall, welcher nicht der Testfallstruktur entspricht	28
4.7	Ein von ChatGPT-4 von OpenAI (2025) erstellter Testfall, welcher nicht kohärente Testschritte besitzt.	29
4.8	Ein von ChatGPT-4 von OpenAI (2025) erstellter Testfall, welcher der Testfallstruktur entspricht und kohärente Testschritte besitzt	31

Gender-Hinweis

In dieser Bachelorarbeit aus Gründen der besseren Lesbarkeit das generische Maskulinum verwendet. Dabei gilt die männliche Sprachform für alle Geschlechter.

Abstract

This bachelor thesis investigates the use of the LLM ChatGPT-4 by OpenAI (2025) for the creation of requirement test cases, taking into account the requirements specifications provided to it. The aim of the work and the evaluation of the results was to find out whether ChatGPT-4 by OpenAI (2025) is able to generate requirement test cases that are of high quality in terms of user-friendliness, error detection, maintainability and reliability and at the same time have the common test case structure according to ISO/IEC/IEEE 29119-1:2022-01 (2022). In addition, these test cases should only contain logical and factually correct test steps. To check their correctness, they were compared with manually created test cases that were created from the same requirement specifications. The intention was to contribute to the improvement of test case generation.

The results of this bachelor thesis show that

Keywords: AI, LLM, RAG, Prompting, Test Case Generation

Zusammenfassung

Diese Bachelorarbeit erforscht den Einsatz des LLM ChatGPT-4 von OpenAI (2025) bei der Erstellung von Anforderungstestfällen unter Berücksichtigung ihrer zur Verfügung gestellten Anforderungsspezifikationen. Ziel der Arbeit und der Auswertung der Ergebnisse war es herauszufinden, ob ChatGPT-4 von OpenAI (2025) dazu in der Lage ist, Anforderungstestfälle zu erzeugen, welche in Bezug auf Benutzerfreundlichkeit, Fehlererkennung, Wartbarkeit und Zuverlässigkeit eine hohe Qualität aufweisen und dabei gleichzeitig die gängige Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) besitzen. Außerdem sollen diese Testfälle nur logische sowie faktisch richtige Testschritte enthalten. Um die Korrektheit zu überprüfen wurden sie mit manuell erstellten Testfällen verglichen, welche aus den gleichen Anforderungsspezifikationen erstellt wurden. Intention war es, einen Beitrag zur Verbesserung der Testfallgenerierung zu leisten.

Die Ergebnisse dieser Bachelorarbeit zeigen, dass

Stichwörter: KI, LLM, RAG, Prompting, Testfallgenerierung

1 Einleitung

Kapitel 1 widmet sich zunächst der Darstellung der Motivation, die der Bachelorarbeit zugrunde liegt und erläutert die Zielsetzung der Untersuchung. Dabei wird verdeutlicht, warum das Thema relevant ist und welchen Beitrag die Arbeit in diesem Kontext leistet. Darüber hinaus werden in diesem Kapitel die zentrale Forschungsfrage formuliert und die methodische Vorgehensweise beschrieben, mit der die gestellten Ziele erreicht werden. Diese Erläuterungen dienen dazu, den inhaltlichen und methodischen Rahmen der Arbeit klar zu definieren.

1.1 Motivation

In den vergangenen Jahren hat in dem Bereich Künstliche Intelligenz (KI) eine rasante Entwicklung stattgefunden. Diese hat dabei eine Auswirkung auf das private als auch auf das geschäftliche Leben. Der Artikel des Branchenverbandes der deutschen Informations- und Telekommunikationsbranche - Bitkom - stellt eine von ihnen durchgeführte Studie vor. Laut dieser Studie hat sich der Anteil der deutschen Unternehmen, welche KI nutzen, von 9 % im Jahr 2022 auf 15 % im Jahr 2023 nahezu verdoppelt. Dabei sehen 68 % der Befragten KI als Chance. (Streim und Hecker, 2023)

In einer Pressemitteilung vom 25. November 2024 gibt das Statistische Bundesamt bekannt, dass bereits 20 % der in Deutschland ansässigen Unternehmen KI verwenden. Dabei sind die drei häufigsten Einsatzbereiche für KI Textmining - Analyse geschriebener Sprache mit 48 %, Spracherkennung mit 47 % und die Erzeugung von natürlicher Sprache mit 34 %. Dies zeigt eine rasante Steigerung der Nutzung von KI alleine in den letzten drei Jahren und es lässt sich erkennen, dass KI bereits vielfältig einsetzbar und nutzbar ist. Gerade in der Informatik und in dessen Teilgebiet der Softwareentwicklung trifft dies zu. (Statistisches Bundesamt, 2024)

In dieser Arbeit stehen KI-Sprachmodelle im Vordergrund. Diese sogenannten Large Language Models (LLM) bieten sich dafür an, redundante und sich ähnliche Aufgaben für den Anwender zu erledigen. LLM werden deshalb bereits in verschiedensten Bereichen der Textgenerierung verwendet. Zu nennen sind dabei Übersetzung, Texterstellung, Textzusammenfassung, Kategorisierung und Klassifizierung. Eine Anwendungsmöglichkeit der Textgenerierung ist deshalb die Testfallgenerierung durch die KI. Das Erstellen von Testfällen mithilfe von KI kann erhebliche Zeit- und Kosteneinsparungen für Unternehmen und Tester ermöglichen. (Kerner, 2024)

Ein Testfall hat eine Struktur, welche definierten Regeln entspricht und in der Softwareentwicklung, aber auch allgemein in fast allen beruflichen und wissenschaftlichen Bereichen wird getestet. Somit ist die Testfallgenerierung durch KI nahezu universell einsetzbar. Es gibt bereits KI-Tools und Arbeiten, welche sich mit dem Thema mit Hilfe von KI erzeugte Testfälle beschäftigt haben. Zu nennen sind dabei die Werke von. Vorteile von KI generierten Testfällen sind dabei eine gesteigerte Qualität und Kosten und Zeiteinsparungen. (Bozic, 2022; Weingartz und Suleymanov, 2024)

Wie bei allen neuen Technologien ist es essenziell, sich auch der damit verbundenen Risiken bewusst zu sein. Ein Nachteil der KI ist zum Beispiel das Phänomen der Halluzination. Dieser Begriff beschreibt das Phänomen, das eine KI Antwort nicht mit dem gegebenen Input übereinstimmt und faktisch nicht richtig ist. (Siebert, 2024)

Ebenfalls ist der Umgang mit Daten nicht sicher. Ein mögliches Problem sind unternehmensinterne und auch persönliche Daten. Es gibt keine Transparenz darüber, wie diese behandelt werden, wenn sie einem LLM zur Verfügung gestellt werden. Persönliche Daten könnten missbraucht werden und LLM könnten mit unternehmensinternen Daten antrainiert werden, welche die Konkurrenz dann nutzen könnte. Daher stellt sich die Frage, ob es sinnvoller ist, ein eigenes Modell anzutrainieren, um die Sicherheit und Kontrolle über die Daten zu gewährleisten. Eine empfohlene Maßnahme für dieses Risiko ist die Anonymisierung persönlicher und sensibler Daten, bevor man diese der LLM zur Verfügung stellt. (Möllers, 2024)

Da die im letzten Absatz beschriebene Lösung mit Zeit und Aufwand verbunden ist, bieten sich andere Methoden an und Modelle, welche Retrieval Augmented Generation (RAG) verwenden können, können sich gut für die Aufgabe der Testfallgenerierung eignen. Die Technik RAG beschreibt den Vorgang, dass das Wissen, welches von der LLM benötigt wird, nicht aus dem Prompt - der Eingabe - kommen muss. Informationen aus Dateien, die dem LLM zur Verfügung gestellt werden, können ebenfalls verwendet werden. (Honroth et al., 2024)

1.2 Forschungsfrage

Die folgende Forschungsfrage, die aus der zugrunde liegenden Motivation abgeleitet wurde, bildet die Grundlage für die Durchführung der Arbeit.

Ist es möglich, dass das LLM ChatGPT-4 von OpenAI (2025) dazu in der Lage ist, aus Anforderungsspezifikationen Anforderungstestfälle zu erzeugen, die eine hohe Qualität in Bezug auf Benutzerfreundlichkeit, Fehlererkennung, Wartbarkeit und Zuverlässigkeit aufweisen, während sie gleichzeitig die gängige Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) besitzen und nur logische sowie faktisch richtige Testschritte enthalten?

Diese Frage zielt darauf ab, herauszufinden, wie gut eine frei verfügbare LLM wie ChatGPT-4 von OpenAI (2025) in der Lage ist, Testfälle zu generieren und ob somit Entwickler und Unternehmen wertvolle Zeit und Ressourcen sparen können, wenn die KI redundante Aufgaben wie das Erstellen und Anpassen von Anwendungstestfällen übernehmen kann und die Ergebnisse nur überprüft werden müssen. Um diese Frage zu beantworten, werden Auswertungsmetriken entwickelt, anhand derer die von ChatGPT-4 von OpenAI (2025) erstellten Testfälle ausgewertet werden können. Außerdem wird die gängige Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) definiert und die faktische Korrektheit der Testfälle damit überprüft, in dem sie mit manuell erstellten Testfällen verglichen werden, welche aus der gleichen Anforderungsspezifikation erstellt wurden.

1.3 Ziel der Arbeit

Ziel der Arbeit ist es, die Grundlagen und Anwendungsmöglichkeiten und -verfahren zu erforschen, durchzuführen und zu evaluieren. Diese Bachelorarbeit soll eine Vorgehensweise entwickeln, die sich für den Einsatz von KI für die Generierung von Anforderungstestfällen aus der KI zur Verfügung gestellten Anforderungsspezifikationen eignet. Diese stützt sich dabei auf bereits bestehender Vorgehensweisen und Modelle. Durch die theoretische und praktische Aufarbeitung, Erprobung und Verbesserung bestehender Vorgehensweisen und Modellen soll sie dabei einen Beitrag leisten, die Effizienz und Qualität der mit Hilfe von KI erstellten Testfälle zu verbessern.

1.4 Vorgehensweise

Das Vorgehen in der vorliegenden Arbeit erfolgt in mehreren Abschnitten. Zunächst wird eine Literaturrecherche durchgeführt. Dabei liegt der Fokus auf dem Finden von bereits vorhandener Vorgehensweisen und Modellen im Bezug auf Textgenerierung im Allgemeinen und Testfallgenerierung im Besonderen. Es ist ebenfalls notwendig, sich über Begriffe und Techniken, welche während der Umsetzung der Arbeit notwendig sind, tiefer zu ergründen. Wichtig ist dabei, eine festgelegte Struktur zu finden oder zu definieren, wie ein Testfall und eine Anforderungsspezifikation aufgebaut sind, welche Merkmale und welche Kriterien sie erfüllen müssen. Es wird recherchiert, was eine KI und was ein LLM ist und warum sich ChatGPT-4 von OpenAI (2025) als LLM für die Testfallgenerierung eignet. Das Ermitteln geeigneter Prompting Methoden sowie der Methode RAG ist ebenfalls erforderlich. Das Ziel ist dabei die Untersuchung und Entdeckung bereits vorhandener Anwendungsmöglichkeiten, Vorgehensweisen und Modellen zur ChatGPT-4 von OpenAI (2025) gesteuerten Testfallgenerierung, anhand derer die Testfallgenerierung in dieser Arbeit ausgeführt werden kann.

Nachdem das passende das Vorgehen ermittelt wurde und passende Testdaten, welche sich in Anforderungsspezifikationen und aus diesen manuell erstellte Anforderungstestfälle gliedern, gefunden werden, beginnt die praktische Umsetzung der Arbeit. Dabei liegt die Durchführung, Dokumentation und Auswertung der von ChatGPT-4 von OpenAI (2025) erstellten Anforderungstestfälle im Vordergrund.

Zum Schluss werden die durch ChatGPT-4 von OpenAI (2025) erstellten Anforderungstestfälle auf ihre Benutzerfreundlichkeit, Fehlererkennung, Wartbarkeit und Zuverlässigkeit überprüft und eingeordnet. Die Einhaltung der definierten Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) ist entscheidend. Die faktische Korrektheit spielt eine ebenso große Rolle. Sie wird überprüft, indem die von ChatGPT-4 von OpenAI (2025) generierten Testfälle mit manuell erstellten Anforderungstestfällen verglichen und auf ihre faktische Korrektheit und Logik gemessen werden. Die Ergebnisse werden analysiert und hinsichtlich ihrer Benutzerfreundlichkeit, Fehlererkennung, Wartbarkeit und Zuverlässigkeit ausgewertet. Dabei werden für die Auswertung Metriken verwendet, die in Kapitel 4 näher erläutert werden.

2 Grundlagen und Hintergrund

Das Kapitel 2 Grundlagen und Hintergrund widmet sich den wissenschaftlichen Grundlagen und klärt über Begriffe und Methoden auf, die für das Verstehen und das Durchführen der Arbeit essenziell sind. In diesem Kapitel wird erst der Begriff KI erklärt, welche wichtigen Teilbereiche es gibt. Anschließend wird der Begriff LLM verwendet, da das LLM ChatGPT-4 von OpenAI (2025) für das Erstellen der Testfälle verwendet wird. Danach wird beschrieben, was Prompting und was RAG ist. Wichtig ist ebenfalls zu klären, wie ein Testfall aufgebaut und was eine Anforderungsspezifikation ist, um die von ChatGPT-4 von OpenAI (2025) erstellten Anforderungstestfälle auswerten zu können.

2.1 KI

In dieser Arbeit wird mithilfe von KI, Anforderungstestfälle erzeugt. Da KI ein weitverbreiteter Begriff ist und es viele Unterbereiche der KI gibt, ist es wichtig, dass hier erst ein grober Überblick über den Begriff KI verschafft wird. Danach werden einzelne Unterbereiche der KI vorgestellt und kurz beschrieben.

2.1.1 Allgemein

Der Begriff KI beziehungsweise im Englischen Artificial Intelligence (AI) beschreibt ein Teilgebiet der Informatik. Dabei gibt es zwei Typen von KI. Erstens die schwache KI, welche auch als enge KI bezeichnet wird. Diese Art der KI kann nur einzelne und beschränkte Aufgaben lösen. Der Begriff KI wurde bereits im Jahr 1956 als Konzept vorgeschlagen. Seitdem entwickelte sie sich immer weiter. Im Jahr 1997 verliert der damalige Schachweltmeister Garry Kasparov gegen den Schachcomputer Deep Blue von IBM und im Jahr 2016 besiegt der Computer AlphaGo von Google Lee Se-dol, ein Go-Meister aus Südkorea. Diese Erfolge zeigen die Entwicklung der KI, sind jedoch auf die schwache KI beschränkt. (Hecker et al., 2018)

Als Zweites gibt es das Konzept der starken KI, welche auch als allgemeine KI bekannt. Folgende Grafik zeigt, wie eine starke KI ihre Umgebung wahrnimmt.

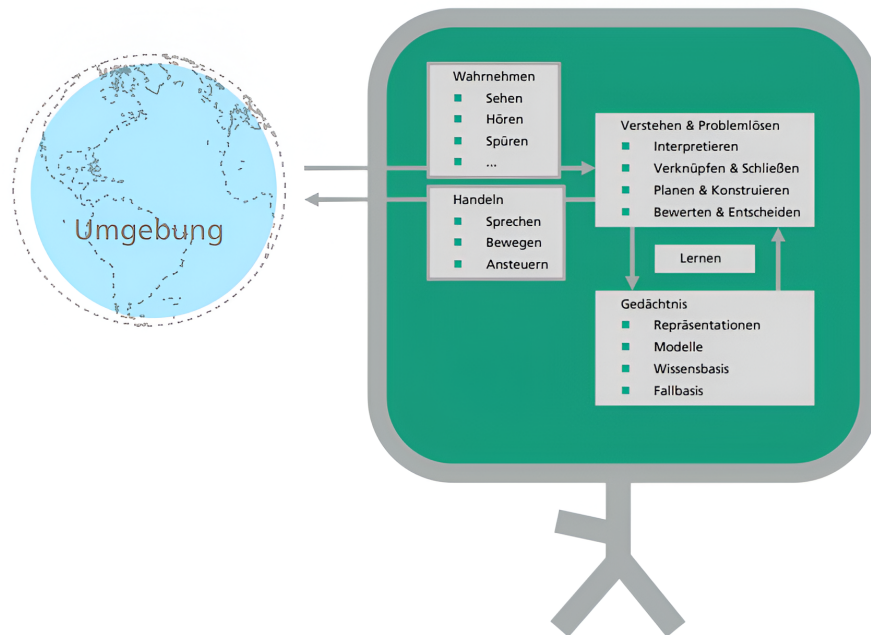


Abbildung 2.1: Die Grafik von Hecker et al. (2018) zeigt die Interaktion eines KI-Systems mit seiner Umwelt

Das Konzept hinter dieser KI besteht darin, dass Maschinen Fähigkeiten verwenden beziehungsweise imitieren können, die menschlich sind. Laut Hecker et al. (2018) bilden sie kognitive Fähigkeiten ab. Es scheint dabei, dass die Maschinen sich selbst und ihre Umgebung wahrnehmen und interpretieren können. Ihre Wahrnehmung und ihre Interpretation speist sich dabei auf Daten, Fakten, konkreten Modellen und Regeln, welche ihr nächstes Handeln bestimmen. Während des Nutzens lernen die Maschinen dabei kontinuierlich weiter, so als würden sie ein Gedächtnis besitzen. (Bhatt et al., 2021; Zhu et al., 2021)

Hecker et al. (2018) nennt noch eine dritte Form der KI, die sogenannte Superintelligenz, diese wird es jedoch in nächster Zeit nicht geben und somit wird in dieser Arbeit nicht auf diese eingegangen.

2.1.2 Bereiche der KI

Nachdem erklärt wurde, um was es sich bei KI handelt, ist es nun wichtig zu erklären, welche Bereiche und Teilgebiete es von KI gibt. Zu nennen sind dabei maschinelles Lernen, Neuronale Netze und Deep Learning.

Folgende Grafik zeigt anschaulich die einzelnen Teilgebiete der KI.

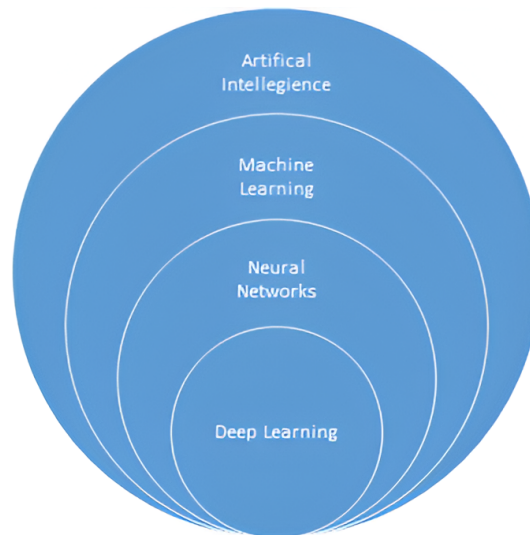


Abbildung 2.2: Die Grafik von Mocko und Bitton-Bailey (2021) zeigt die Beziehung zwischen KI, maschinellem Lernen, neuronalen Netzen und Deep Learning

KI wird bereits in Kapitel 2.1 beschrieben und umfasst andere Bereiche. Die Grafik ist dabei so zu verstehen, dass der größere Kreis die jeweils kleineren Kreise umfasst, das heißt maschinelles Lernen ist ein Teil von KI, Neuronale Netze sind ein Teilgebiet von KI und von maschinellem Lernen und Deep Learning ist ein Teilgebiet von KI, von maschinellem Lernen und von Neuronalen Netzen. (Bhatt et al., 2021; Zhu et al., 2021)

Im Folgenden werden nun einige wichtige Bereiche der KI aufsteigend vom kleinsten zum größten Teilbereich erläutert.

Deep Learning:

Deep Learning ist ein Teilgebiet der Neuronalen Netze. Sie sind dadurch gekennzeichnet, dass sie ein Hidden Layer besitzen, welches aus mehr als einer Neuronenschicht besteht (Bhatt et al., 2021); (Zhu et al., 2021). Folgende Grafik zeigt anschaulich ein Neuronales Netz und ein Deep Learning Netz.

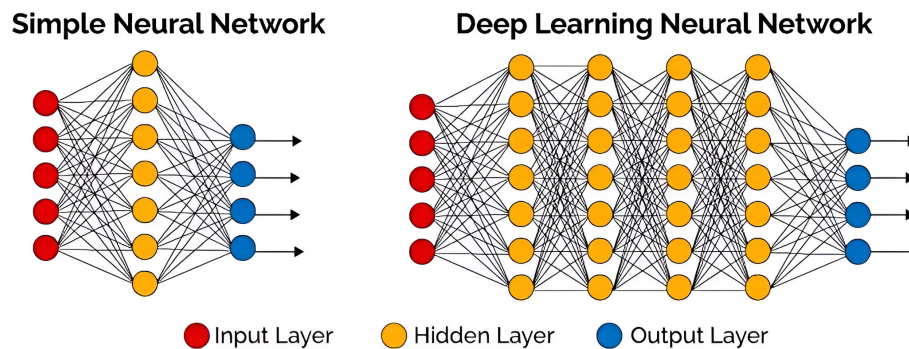


Abbildung 2.3: Die Grafik von Dymatrix (2018) zeigt ein einfaches Neuronales Netz und ein Deep Learning Netz

Sie können dabei Millionen Neuronenschichten beinhalten und dadurch immer komplexere Aufgaben lösen. Dies ist eine Abhandlung über das Thema KI. (Bhatt et al., 2021; Zhu et al., 2021)

Weitere wichtige KI-Bereiche wie LLM, Prompting und RAG werden in den Kapiteln 2.2, 2.3 und in Kapitel 2.4 erläutert, da diese eine wichtige Rolle in dieser Arbeit spielen.

Neuronale Netze:

Bei neuronalen Netzen handelt es sich um ein Teilgebiet des maschinellen Lernens. Idee der neuronalen Netze ist es, menschliche Neuronen nachzubilden und somit menschliches Denken nachzuahmen. Ein neuronales Netz besteht dabei aus Knoten, die aneinandergereiht sind. Sie haben eine Eingabeschicht, ein Hidden Layer, welches aus einer Neuronenschicht und einer Ausgabeschicht besteht, die miteinander vernetzt sind. Sie trainieren ebenfalls in Schleifen und Durchgängen und werden durch das Wiederholen besser und schneller. (Bhatt et al., 2021; Zhu et al., 2021)

Maschinelles Lernen:

Maschinelles Lernen auf Englisch Machine Learning ist ein Teilgebiet der KI. Es beschreibt das Verfahren, in welchem Maschinen durch das Wiederholen von Aufgaben lernen. Dabei gibt es Algorithmen, welche mit Daten gefüttert werden und der Lernprozess ist eine Schleife, in welcher dieser wiederholt wird. Dadurch lernt die Maschine die Aufgabe zu lösen. Dies geschieht, in dem es Feedback erhält und so lernt, wie es die Aufgabe richtig löst. Es wird dabei kein Lösungsweg vorgegeben wie bei herkömmlichen Algorithmen, sondern die Maschine lernt durch Versuch und Irrtum beziehungsweise auf Englisch durch Trial and Error. (Bhatt et al., 2021; Zhu et al., 2021)

2.2 Large Language Model (LLM)

In dieser Arbeit wird die LLM ChatGPT-4 von OpenAI (2025) verwendet, um Testfälle aus Anforderungsspezifikationen zu generieren. Der Begriff LLM steht für Large Language Model, welches als großes Sprachmodell übersetzt werden kann. Über die Eigenschaften und Entwicklungen von LLM klärt dieses Kapitel auf. Dies ist wichtig, um zu verstehen, weshalb in dieser Arbeit die LLM ChatGPT-4 von OpenAI (2025) genutzt wird und wieso dies sinnvoll ist.

Diese LLMs stellen eine Kombination aus zwei Bereichen der KI dar. Sie sind ein Teilgebiet von Natural Language Processing (NLP). Dieser Prozess beschreibt die Möglichkeit der Verarbeitung natürlicher Sprache. Dabei wird dieser mithilfe der Trainingsmöglichkeit, welche Deep Learning bietet, kombiniert. (Chang et al., 2023; Kaddour et al., 2023; Naveed et al., 2024)

Folgende Grafik zeigt anschaulich, in welchem Gebiet LLM innerhalb der KI angesiedelt sind:

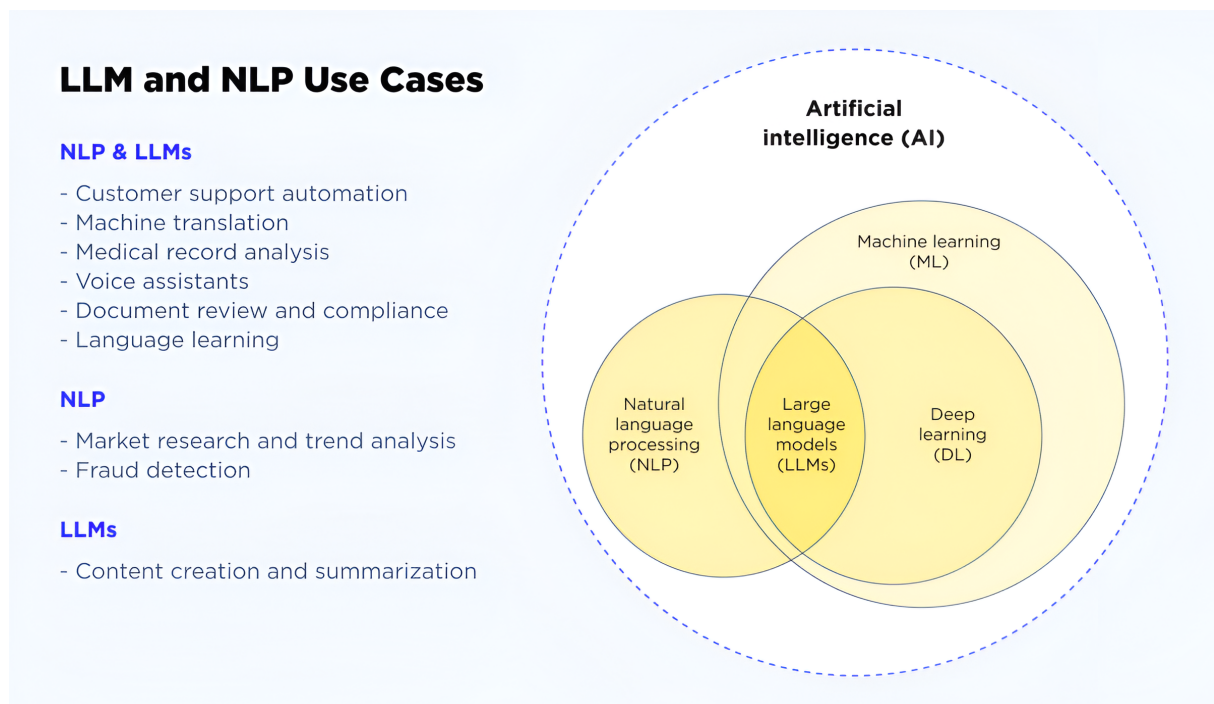


Abbildung 2.4: Die Grafik von Hodiak (2024) zeigt LLM und NLP und verschiedene Anwendungsfälle der Technologien

Dabei können diese Modelle durch das Training und der Verfügbarkeit von großen Datenmengen natürliche Sprache nachahmen. Ein bekanntes Beispiel für LLM sind Chatbots wie ChatGPT-4 von OpenAI (2025). Diese LLM können dabei auf verschiedenste Aufgaben entwickelt und trainiert werden und sind somit vielseitig einsetzbar. Häufige Aufgaben sind laut Kerner (2024) die bereits erwähnten Chatbots, Textgenerierung und auch die Forschung und Entwicklung. Dabei können LLM auf vielen Sprachen arbeiten, welche sie nahezu universell einsetzbar machen. (Chang et al., 2023; Kaddour et al., 2023; Naveed et al., 2024)

2.2.1 Entwicklungen im Bereich LLM

Aufgrund des Erfolgs von LLMs, ihren Möglichkeiten und ihren vielseitigen Einsatzmöglichkeiten sind sie dynamisch und es gibt viele LLMs. Neue Modelle können selbst entwickelt werden. Vorhandene Modelle können auch antrainiert werden, was als Finetuning bezeichnet wird. Ein Werk, welches sich ausführlich mit dem Thema LLM erstellen und finetunen beschäftigt ist Géron (2019). Das Erzeugen und das Finetuning von LLM benötigt jedoch Zeit, Ressourcen und eine Menge an Daten. Deshalb ist es effizienter und präziser, sich bereits bestehende Modelle anzusehen, welche es bereits in großer Anzahl gibt. Eine bekannte Open Source Plattform, in welcher nach LLM gesucht und in welcher diese heruntergeladen werden können, ist die Website Hugging Face (2025). Sie bietet eine große Auswahl an LLM an. Während am 16.01.2025 1,295,659 verschiedene Modelle angeboten wurden, wurden etwas mehr als zwei Wochen später am 04.02.2025, bereits 1,378,895 verschiedene Modelle angeboten, dies zeigt die enorme Vielfaltigkeit und die rapide Entwicklung im Bereich LLM, in welcher im Minutentakt neue Modelle hinzukommen. (Géron, 2019)

2.3 Prompting

Die Eingabe bzw. der Prompt ist ein wesentlicher Bestandteil zum Nutzen von KI. Jede Eingabe ist ein Prompt und die Qualität des Prompts kann über die Qualität der Antwort, welche das LLM liefert, entscheiden. Deshalb ist es wichtig, diesen Begriff und die Vorgehensweise, nach welcher in dieser Arbeit vorgegangen wird, zu erklären. Der englische Begriff Prompting beziehungsweise Prompt kann als Eingabeaufforderung übersetzt und verstanden werden. Mithilfe eines Prompts kann der Nutzer der LLM eine Aufgabe oder eine Mitteilung liefern, welche diese verarbeiten und dann lösen kann. Prompting ist das entscheidende Schlüsselement, um mit LLM zu interagieren.

Folgender Screenshot zeigt einen Prompt und die Antwort der LLM in diesem Fall von ChatGPT-4 von OpenAI (2025), welcher von Müller (2025) dem Autor dieser Arbeit erstellt wurde:

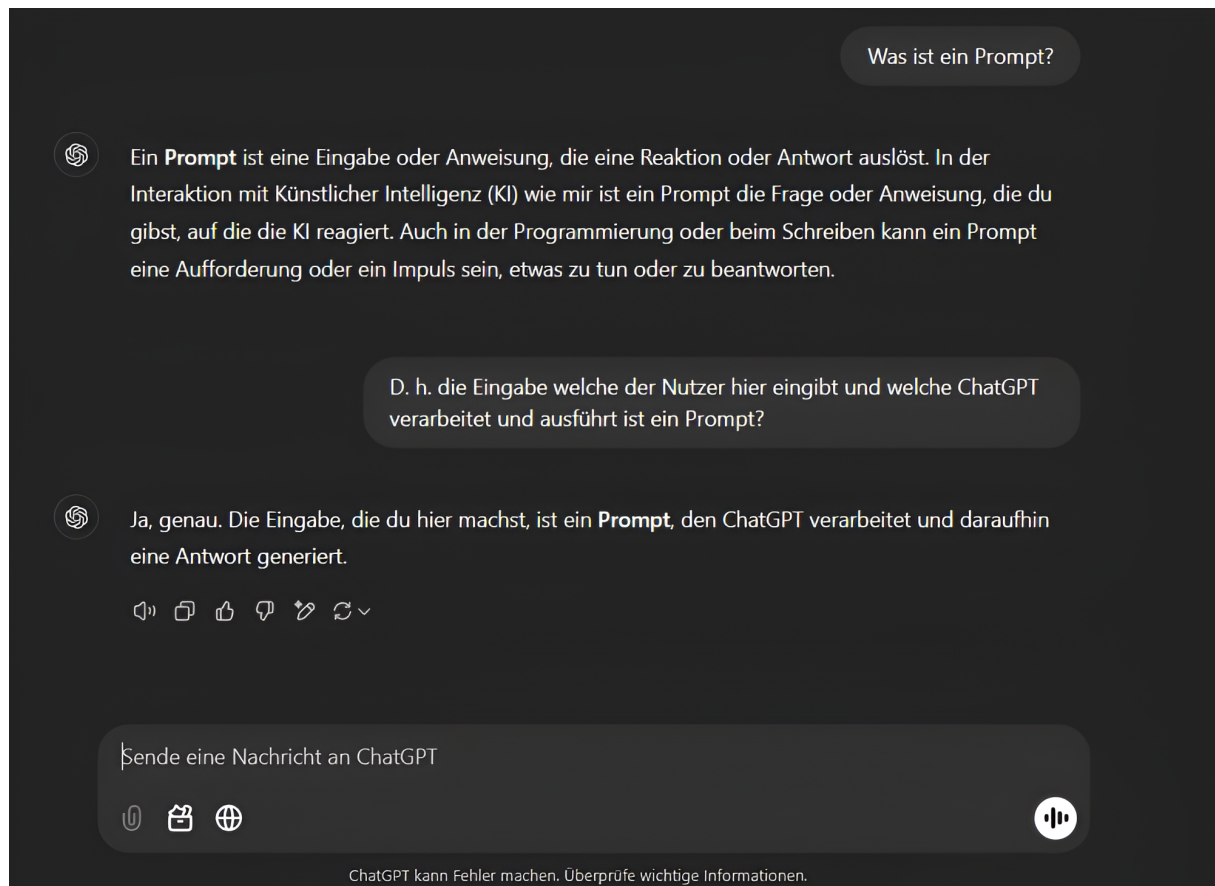


Abbildung 2.5: Ein von Müller (2025) dem Autor dieser Arbeit erstellter Screenshot eines Beispiels für einen Prompt in ChatGPT-4 von OpenAI (2025) in der Benutzeroberfläche

Prompts können dabei einfache Sprachaufforderungen sein wie anhand des in der Grafik gezeigten Beispiels. Jedoch können diese auch komplexer sein und für vielseitige Aufgaben unterschiedlich genutzt werden. Dabei gibt es den Begriff Prompt Engineering und verschiedene Prompt-Techniken, welche in Kapitel 2.3.1 erklärt werden. (J. Gu et al., 2023; P. Liu et al., 2023)

2.3.1 Prompt Engineering und Prompt-Techniken

Unter dem Begriff Prompt Engineering versteht man das gezielte Nutzen von Prompts, um bessere Ergebnisse zu erhalten. Dabei wird systematisch vorgegangen, um ohne das Finetuning von LLM konkretere und verbesserte Lösungen von diesen zu erhalten. Es gibt viele Prompt-Techniken. Eine Prompt-Technik, welche sich flexibel an unterschiedlichste Aufgaben anpassen kann, ist das sogenannte Soft Prompting. In diesem werden Parameter, welche lernen können, direkt in die Eingabevektoren des Modells eingebettet. Dadurch müssen die Prompts nicht als Text eingegeben werden. Es gibt noch viele weitere Prompt-Techniken deren Aufzählung und Erklärung zu ausführlich sind. Wichtig ist zu verstehen, dass Prompting flexibel ist und es viele Möglichkeiten gibt. Diese sind für diese Arbeit nicht relevant, da für diese Arbeit eine spezielle Technik verwendet wird, welche in Kapitel 2.4 erklärt wird. (J. Gu et al., 2023; P. Liu et al., 2023)

2.4 Retrieval Augmented Generation (RAG)

Bereits in Kapitel 1.1 wird RAG erwähnt. Die Technik RAG steht für Retrieval Augmented Generation und ermöglicht es, einem LLM Dokumente zur Verfügung zu stellen, in welcher sich Wissen befindet. In dieser Arbeit kann dem LLM per RAG eine Anforderungsspezifikation zur Verfügung gestellt werden, in welcher die Anforderungen zu finden sind, aus welchen das LLM die Anforderungstestfälle generieren soll. Deshalb ist RAG für diese Arbeit von besonderer Wichtigkeit. RAG ist eine Technik, in welcher einem LLM Dokumente zur Verfügung gestellt werden. Dabei werden die in den Dokumenten enthaltenen Informationen für das LLM nutzbar gemacht. Die Dokumente und Anfragen werden dabei in kleine Abschnitte, welche als Chunks bezeichnet werden, aufgeteilt. RAG lässt sich dabei in vielen LLM anwenden und optimiert diese, ohne dass diese mit viel Aufwand trainiert werden müssen. (Gao et al., 2024; Wu et al., 2024)

Folgende Grafik zeigt anschaulich, wie RAG funktioniert:

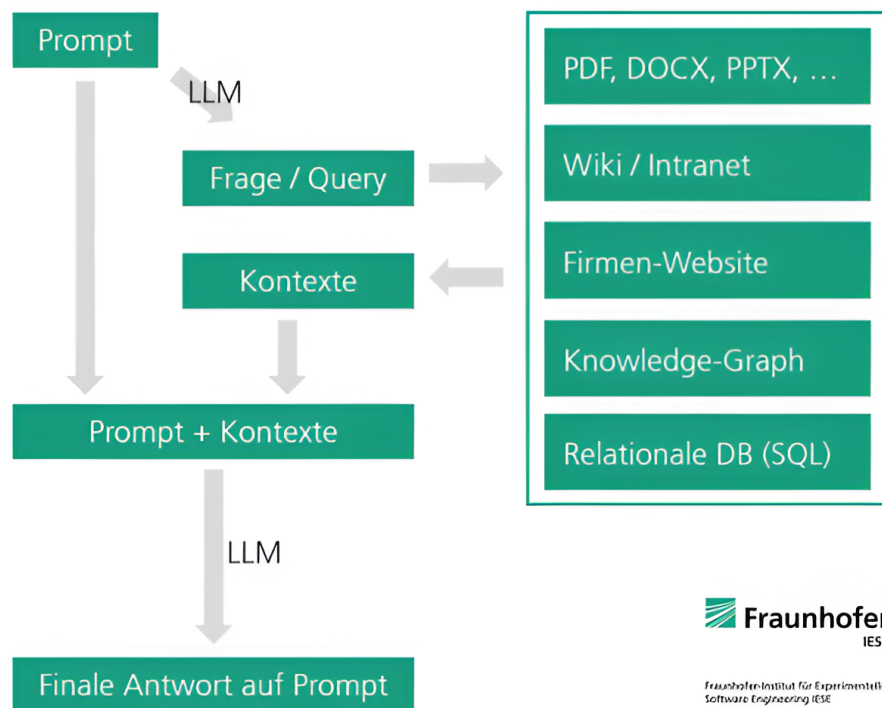


Abbildung 2.6: Die Grafik von Honroth et al. (2024) zeigt, wie Retrieval Augmented Generation (RAG) im Detail funktioniert

Im Folgenden werden die einzelnen Schritte von RAG erklärt, dabei gibt es laut Schmid und Sommer (2024) und laut Miesle (2023) 5 Schritte. In dieser Arbeit werden die Begriffe so verwendet, wie sie Schmid und Sommer (2024) verwendet haben, da es sich um ein Werk auf Deutsch handelt und diese Arbeit ebenfalls auf Deutsch ist:

1. Indexierung:

Die Indexierung beschreibt den Prozess, wie die Daten aus einem Dokument in kürzere Abschnitte den Chunks aufgeteilt werden, um die Daten für die LLM verwertbar zu machen. Die Chunks werden maschinenlesbar gemacht und in einer Vektordatenbank gespeichert, aus welcher sie nun für die LLM zugreifbar sind. Sie werden vektorisiert.

2. Benutzereingabe:

Die Benutzereingabe erfolgt als Prompt in natürlicher Sprache, welche sich auf die im Dokument befindlichen Daten beziehen.

3. Retrieval:

Nachdem die Maschine die Eingabe erhalten hat und diese ebenfalls vektorisiert hat, wird mit der Methode des nächsten Nachbarn die höchste Ähnlichkeit zwischen der Eingabe und der in der Vektordatenbank gespeicherten Chunks priorisiert.

4. Augmentation:

Der LLM wird diese Kombination aus Prompt des Benutzers und der Kontext zu Verfügung gestellt.

5. Generation:

Es wird nun eine Antwort generiert, welche sowohl die erhaltenen Informationen als auch das Training des jeweiligen LLM kombiniert.

RAG ist vielfältig einsetzbar und eignet sich in vielen Bereichen. Die Autoren (Gao et al., 2024; Lewis et al., 2021; Salemi & Zamani, 2024) kommen zu dem Schluss, dass durch RAG LLMs bessere Erlebnisse liefern können. Ergebnisse, welche RAG benutzen, zeichnen sich bei ihrer Performance aus und sind spezifischer und faktenreicher als Ergebnisse, die nur von der LLM ohne RAG erzeugt wurden. (Gao et al., 2024; Lewis et al., 2021; Salemi und Zamani, 2024)

2.4.1 Einsatzmöglichkeiten von RAG

RAG spielt in dieser Arbeit eine entscheidende Rolle. Mithilfe von RAG kann der LLM eine Anforderungsspezifikation zur Verfügung gestellt werden, aus welcher bereits manuell Anforderungstestfälle generiert wurden. Durch RAG kann die LLM diese Anforderungsspezifikation, welche eine PDF-Datei ist, gut verarbeiten und daraus Informationen ziehen. Dabei kann sie Informationen in Form von Text und von Bildern verarbeiten und berücksichtigen. Dies kann getrennt von einander geschehen, also PDF-Dateien sein, welche entweder nur Text oder Bilder enthalten, oder PDF-Dateien, welche Text und Bilder beinhalten. (Gao et al., 2024; Wu et al., 2024)

2.5 Testfall

Es ist entscheidend zu definieren, was ein Testfall ist, welche Grundlagen und Definitionen für dessen Aufbau gegeben sind. Ohne dieses Wissen ist es unmöglich, eine Metrik für die Auswertung und Qualität der generierten Testfälle zu definieren. Somit ist es ohne dieses Kapitel unmöglich, die von ChatGPT-4 von OpenAI (2025) generierten Testfälle auszuwerten und die Forschungsfrage zu beantworten.

In dieser Arbeit werden Testfälle mithilfe von der LLM ChatGPT-4 von OpenAI (2025) erzeugt. Deshalb ist es wichtig zu beschreiben, was ein Testfall ist, wie er aufgebaut ist und welche Merkmal diese besitzen. Ein Testfall, ist wie bereits der Name sagt, ein Test-szenario, in welchem getestet wird, ob die Anwendung die Funktionalitäten so erfüllt, wie sie erwartet werden. Für Testfälle gibt es eine internationale und standardisierte Definition und ein Regelwerk von ISO/IEC/IEEE 29119-1:2022-01 (2022). Eine Übersicht, welche die Testbegrifflichkeiten genau erläutert, findet sich in dem ISTQB/GTB Standardglossar der Testbegriffe (2017). Dieses Standardglossar beschreibt dabei sämtliche Begrifflichkeiten auf Englisch und ihrer offiziellen Übersetzung auf Deutsch.

2.5.1 Aufbau eines Testfalls

Ein Testfall besitzt einen Identifier, einen Namen, eine Beschreibung, Vorbedingungen, welche erfüllt sein müssen, damit der Testfall durchgeführt werden kann und Ziele, die erreicht werden sollen ISO/IEC/IEEE 29119-1:2022-01 (2022). Folgende Grafik zeigt ein Testfalldesign für eine SAP-Applikation:

Test Case Titel		Upload external catalog to Ariba Network		
Preconditions		Ariba network users with the required permissions Catalog file in the desired format		
Testobject		Ariba Catalog		
Goals for this Test Case		1. Functional goal: Successfully upload of a catalog file to Ariba Network. 2. Nonfunctional goal: The file can be uploaded within 10 seconds.		

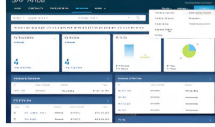
Step	Description (Perform actions and screenshots)	Executable (Transaction, Fiori, URL, etc.)	Possible test data	Expected result (Is required to detect defects)
01	Log in Log in to the system as an external supplier with <username> and <password>.	http://supplier.ariba.com		You will see "cockpit" home site. 
02	Go to Upload external catalog view Select menu <catalogs>  Select button Create default catalog: 			View «details» appears: 
03	Create a new catalog Enter <catalog name> Formulate a <description> Click <save>			Catalog has been successfully uploaded and saved. The status of the catalog is "published".

Abbildung 2.7: Die Grafik von Enderli (2019) zeigt einen Testfall im SAP Solution Manager

Der Testfall gliedert sich in Schritten, welche eine Beschreibung und ein erwartetes Ergebnis besitzen. In den Schritten wird somit überprüft, ob die Anwendung die Anforderungen erfüllt und somit das erwartete Ergebnis eintritt. Testfälle können dabei sowohl positiv als auch negativ sein. Bei negativen Testfällen wird überprüft, ob das erwartete Ergebnis nicht eintritt, da die Voraussetzungen nicht erfüllt sind. (ISO/IEC/IEEE 29119-1:2022-01, 2022; ISTQB/GTB Standardglossar der Testbegriffe, 2017)

2.6 Anforderungsspezifikation

Eine Anforderungsspezifikation ist das Fundament, aus dem Testfälle erstellt werden. Ein Testfall überprüft, ob die Anforderungen aus dieser Spezifikation erfüllt werden. Demnach ist eine Anforderungsspezifikation ein Dokument, welches alle Anforderungen beziehungsweise im Englischen als Requirements bezeichnet, enthält, welche das System erfüllen muss und dessen Erfüllbarkeit und nicht Erfüllbarkeit mit Testfällen abgedeckt und getestet werden. Da sie der Grund für die Testfälle sind, müssen Anforderungsspezifikationen vollständig, präzise und in sich konsistent sein. Anforderungsspezifikationen enthalten System- und Benutzeranforderungen. Systemanforderungen beschreiben dabei die Benutzeranforderungen genau, wobei die Benutzeranforderungen funktionale und nicht funktionale Requirements besitzen. Folgende Grafik veranschaulicht anhand eines Beispiels den Unterschied zwischen funktionalen und nicht funktionalen Requirements:

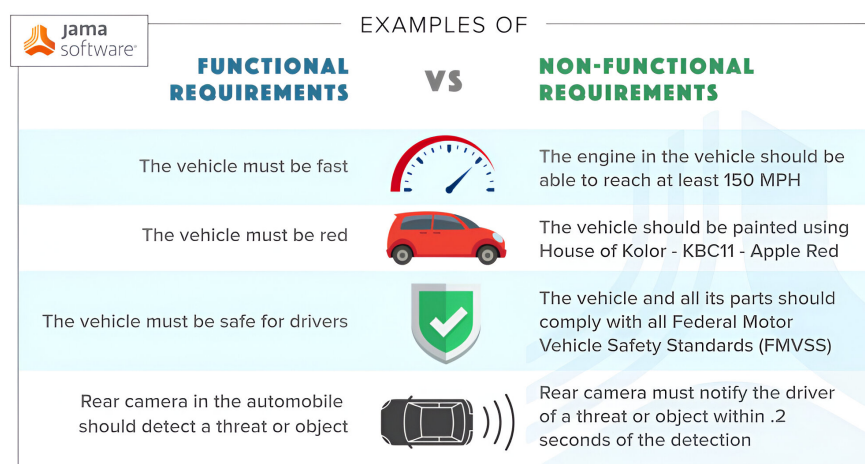


Abbildung 2.8: Die Grafik von Jama Software (2024) zeigt die Eigenschaften von funktionalen vs nichtfunktionalen Requirements

Nicht funktionale Requirements beschreiben laut Visure (2024) zum Beispiel die Bereiche Sicherheit, Wartung und Kriterien an das System. In dieser Arbeit geht es um funktionale Requirements. Diese legen fest, was wie das System funktioniert und was passiert, wenn etwas im Positiven als auch wenn etwas im Negativen passiert. Der Unterschied ist gut an der Grafik von Jama Software (2024) zu erkennen. Während die funktionellen Requirements festlegen, dass das Auto schnell sein und eine rote Farbe besitzen muss, legen die nicht funktionalen Requirements exakt fest, welche Geschwindigkeit das Auto erreichen und welchen genauen Rotton dieses besitzen muss. Funktionale Requirements dienen als Grundlage für die Testfallgenerierung durch ChatGPT-4 von OpenAI (2025) in dieser Arbeit. (Aysolmaz et al., 2018; Barmi et al., 2011; Mustafa et al., 2021)

2.6.1 Bedeutung der Anforderungsspezifikation für die Testfallgenerierung

In vielen Softwareprojekten werden Testfälle durch Requirements erzeugt. Anforderungsspezifikationen dienen als Grundlage für die Testfallgenerierung durch ChatGPT-4 von OpenAI (2025) in dieser Arbeit. Sie werden ChatGPT-4 von OpenAI (2025) per RAG übermittelt und per Prompt dazu aufgerufen, Testfälle zu generieren, welche diese Requirements in Positiv- und Negativfällen abdecken. Dies ist essenziell für diese Arbeit, da es ohne Requirements nicht möglich ist, die von ChatGPT-4 von OpenAI (2025) generierten Testfälle auszuwerten. Mit ihrer Hilfe kann nun jedoch präzise festgestellt werden, wie diese im Bereich Vollständigkeit, Qualität und Fehlerhäufigkeit vorkommen. (Aysolmaz et al., 2018; Barmi et al., 2011; Mustafa et al., 2021)

3 Related Work und aktuelle Forschung

In dieser Arbeit wird die LLM ChatGPT-4 von OpenAI (2025) verwendet, um Anforderungstestfälle zu generieren. Diese Arbeit benutzt somit ein Sprachmodell zur Textgenerierung, da Testfälle in Form eines Textes erstellt werden. Für das Vorgehen bei der Erstellung von Testfällen und für das Aufstellen von Kriterien für die Auswertung der Testfälle in Kapitel 4.4.4 ist es wichtig, wissenschaftliche Arbeiten im Bereich der Textgenerierung und Testfallgenerierung mithilfe von LLMs zu berücksichtigen. In diesem Kapitel werden Arbeiten, welche sich mit diesen Bereichen beschäftigen, erläutert und deren Ergebnisse eingeordnet.

Für den Einstieg zu dem Thema eignet sich ein wissenschaftliches Paper von Iqbal und Qureshi (2022), welches einen Überblick im Bereich Textgenerierung bietet. Dieses Paper stellt viele Deep Learning Modelle vor, die für die Generierung von Text verwendet wurden. Dabei werden die verschiedenen Modelle zusammengefasst und das Paper ermöglicht einen detaillierten Überblick über die Vergangenheit, Gegenwart und Zukunft von Textgenerierungsmodellen im Deep Learning. Die Autoren Iqbal und Qureshi (2022) gehen davon aus, dass in Zukunft immer bessere Modelle entwickelt werden und dass die Forschung im Bereich der Texterzeugung weiter vorangetrieben wird. Das Paper macht jedoch auch deutlich, dass Modelle bei längeren Texten die natürliche Sprache noch nicht vollständig begriffen haben. (Iqbal und Qureshi, 2022)

Diese möglichen Einschränkungen sind für diese Arbeit nicht relevant, da in dieser Arbeit für die Testfallgenerierung keine großen Texte weder für den Input an ChatGPT-4 von OpenAI (2025) noch als Output verwendet wird. Es ist trotzdem wichtig, diese Erkenntnis zu berücksichtigen, wenn es darum geht, die Prompts zu schreiben und der KI die Anforderungsspezifikation zu übermitteln.

Nicht nur die Textgenerierung durch KI ist wichtig, sondern auch die Auswertung des generierten Textes. Yuan et al. (2021) stellten sich die Frage, wie man die generierten Texte nach Flüssigkeit, Genauigkeit und Effektivität beurteilen kann. Dafür entwickelten sie eine Metrik: Der sogenannte „BARTSCORE“ bewertet flexibel und unüberwacht aus unterschiedlichen Blickwinkeln. Unter anderem betrachtet er Informativität, Geläufigkeit oder Faktizität der generierten Texte. Yuan et al. (2021) kommen zum Schluss, dass ihr Modell in 16 von 22 Settings besser abgeschnitten hat als andere gut bewertete Metriken. (Yuan et al., 2021)

Diese Erkenntnis zeigt, wie wichtig es für die Auswertung der Arbeit ist, eine gute Metrik aufzustellen, welche die Qualität des generierten Textes und damit die Qualität der generierten Testfälle auswertet.

Die Werke von (Lazić, 2013; Nirpals & Kale, 2011) liefern einen Überblick über vorhandene Metriken im Bereich des Testens. Die Autoren der beiden Werke (Lazić, 2013; Nirpals & Kale, 2011) kommen zu dem Schluss, dass Metriken wichtig für das Testen sind. (Nirpals und Kale, 2011; Lazić, 2013)

Diese Werke zeigen, dass es im Bereich des Testens viele Metriken und Kriterien gibt, anhand derer die Qualität von Testfällen ausgewertet werden können. Da in dieser Arbeit wenig Mittel zur Verfügung stehen, ist es wichtig, sich auf wichtige Kriterien zu beschränken.

Das Werk von Tran et al. (2025) untersucht die wichtigsten Qualitätskriterien aus der Anwendersicht, da es sehr viele Qualitätskriterien gibt. Die Autoren Tran et al. (2025) kommen dabei zu dem Schluss, dass die Qualitätskriterien Fault Detection (Deutsch: Fehlererkennung), Maintainability (Deutsch: Wartbarkeit), Reliability (Deutsch: Zuverlässigkeit) und Usability (Deutsch: Benutzerfreundlichkeit), die wichtigsten Kriterien für Testfälle sind. Der Stellenwert der Qualitätskriterien ist dabei jedoch abhängig vom Kontext der Tests. Diese Erkenntnis ist eine wichtige Grundlage, um die von ChatGPT-4 von OpenAI (2025) erstellten Testfälle auszuwerten. (Tran et al., 2025)

Das Werk von Wang und Zhu (2024) wirft Fragen im Bereich der Qualität und Korrektheit der von LLM generierten Code auf. Die Autoren Wang und Zhu (2024) gehen davon aus, dass sie Fehler im Code aufspüren können, indem sie Inkonsistenzen finden. Sie kommen zu dem Ergebnis, dass sie in der Lage sind, 75 % der von GPT-4 generierten fehlerhaften Programme zu erkennen, wobei sie eine Falsch Positiv Rate von 8,6 % haben. Wang und Zhu (2024) kommen zu der Erkenntnis, dass Entwickler möglicherweise nur eine kleine Teilmenge des LLM generierten Codes untersuchen müssen, um die meisten Fehler zu finden. Dieses Werk stellt Metriken auf, um Accuracy (Deutsch: Genauigkeit), Precision (Deutsch: Präzision), Recall (Deutsch: Sensitivität) und False Positive Rate (Deutsch: Falsch-Positiv-Rate) von LLM generierten Code auszuwerten. Diese Metrik ist wichtig, um die von Tran et al. (2025) aufgestellten Qualitätskriterien auswerten zu können. (Wang und Zhu, 2024)

Die Metriken von Wang und Zhu (2024) dienen als eine Grundlage, um die von Tran et al. (2025) aufgestellten Qualitätskriterien für das Messen der Qualität der von ChatGPT-4 von OpenAI (2025) erstellten Testfälle in zu messen. In Kapitel 4 wird erläutert, wie diese Metriken aufgebaut sind.

Die Autoren Li et al. (2024) erforschen das Thema Halluzinationen im Bereich der LLM. Das Ziel ist es, Halluzinationen zu erkennen und die Anzahl der Halluzinationen zu reduzieren. Dabei kommen Li et al. (2024) zu dem Schluss, dass die Anzahl der Halluzinationen durch gezieltes Training und RAG verringert werden, jedoch nicht vollständig eliminiert werden kann. Da in dieser Arbeit, dass LLM ChatGPT-4 von OpenAI (2025) verwendet wird, ist es wichtig, das Phänomen der Halluzination zu untersuchen. Die von Li et al. (2024) aufgestellte Methodik zur Messung der Halluzinationsrate (HR) wird in dieser Arbeit in Kapitel 4.4.2 in modifizierter Form verwendet. (Li et al., 2024)

Im Bereich des Testens gibt es unterschiedliche Bereiche, für diese Arbeit ist es relevant herauszufinden, ob bereits KI oder LLMs im Bereich des Testens und zur Testfallerstellung eingesetzt werden. Folgende Arbeiten zeigen den Einsatz von KI oder LLMs im Bereich des Testens.

Das Ziel der Arbeit von Khan et al. (2024) ist es, einen Überblick über die Rolle von KI bei der Automatisierung von Softwaretests zu geben. Dabei stellen die Autoren Khan et al. (2024) die These auf, dass das Ziel der automatisierten Softwaretesterstellung mit KI möglich sein könnte. Im Werk werden zum Schluss entscheidende Schwierigkeiten, Probleme und Voraussetzungen vorgestellt. Am Ende bekräftigen Khan et al. (2024) jedoch ihre These, dass das Testen von Software mit einer Reihe von Automatisierungsgadgets möglich sein könnte. (Khan et al., 2024)

Die Autoren Bhandari et al. (2024) beschäftigen sich mit dem Einsatz und dem Potenzial von LLMs im Bereich des Chip-Testing. Dabei sind funktionale Tests, die sich auf Testbenches stützen, wichtig für das Chipdesign. Es wird Feedback in die LLM eingegeben, um so die Testbench-Generierung zu verbessern. Dies findet iterativ statt, um die Testabdeckung zu verbessern. Bhandari et al. (2024) nennen als Herausforderungen die Wiederholung der Antworten und die Feinabstimmung des LLM mit dem Umfang der Arbeit. Die Wertung der Testbenches in Bezug auf die Abdeckung gestaltet sich als nicht einfach. Trotzdem kommen sie zum Schluss, dass durch das Feedback die Testbenches besser verstanden werden. Ein entscheidender Faktor für die Qualität der Ausgabe ist dabei der Prompt. (Bhandari et al., 2024)

In der Arbeit von Ouédraogo et al. (2024) wird die Wirksamkeit von LLM bei der Erstellung von Unit Tests untersucht. Unit Tests sind ein Framework zur Entwicklung Softwaretests. Dabei kommen Ouédraogo et al. (2024) zu dem Schluss, dass sich die mit LLM erzeugten JUnit Tests mit den von Menschen geschriebenen Tests in Bezug auf Codierungsstandards ähneln. (Ouédraogo et al., 2024)

Die Autoren S. Gu et al. (2024), welche sich in ihrer Arbeit ebenfalls mit dem Einsatz von KI bei der Erstellung von Unit Tests beschäftigen, nennen drei mögliche Probleme bei der Erzeugung von Testfällen. Ein unzureichender Kontext, fehlende Test- und Abdeckungsinformationen und das Problem, dass LLM sich in Wiederholungsschleifen befinden können. Als Lösung bieten sie das Tool TestART an, welches eine Erfolgsquote von 78,55 % aufweist. In einer weiteren Arbeit wird ein Tool namens ChatUniTest vorgestellt, welches ebenfalls die Qualität von durch KI erzeugte Unit Tests verbessern soll. (S. Gu et al., 2024)

Mit dem Thema des automatischen Benutzeroberflächen (GUI)-Testings beschäftigt sich eine weitere Arbeit. Dabei sehen sie einen großen Fortschritt im Bereich des automatischen GUI-Testing, kommen jedoch zu der Erkenntnis, dass von LLM erzeugte Tests noch eine geringe Aktivitätsabdeckung aufweisen. Dies bedeutet, dass nur ein geringer Teil der möglichen Benutzeraktionen und Interaktionen mit der GUI abgedeckt sind. Dadurch kann mit diesen Tests nicht gewährleistet werden, dass alle Funktionen ordnungsgemäß und nach Vorgabe funktionieren. (Z. Liu et al., 2024)

Diese Werke zeigen, dass KI und LLMs bereits in verschiedenen Testbereichen eingesetzt werden. Dabei handelt es sich um die Bereiche der Softwaretests, des Chip-Testing, der Unit Tests und des GUI-Testing. Dies zeigt, dass es möglich ist, dass LLM wie ChatGPT-4 von OpenAI (2025) in der Lage ist, Anwendungstestfälle zu generieren, welche eine bessere Qualität besitzen als manuell erstellte Anwendungstestfälle.

4 Methodik

Kapitel 4 beschreibt die Methodik der Arbeit, es beschreibt das verwendete LLM ChatGPT-4 von OpenAI (2025), die Testdaten welche ChatGPT-4 von OpenAI (2025) zu Verfügung gestellt werden, das Vorgehen und die Prompts an ChatGPT-4 von OpenAI (2025), um die Testfälle zu generieren und die Qualitätskriterien, nach denen der generierten Testfälle ausgewertet und bewertet werden.

4.1 Verwendetes LLM

Dieser Abschnitt beschreibt das verwendete LLM, welche Eigenschaften es besitzt und aus welchen Gründen es für diese Arbeit verwendet wird. Für die Arbeit wird ChatGPT-4 von OpenAI (2025) verwendet. Der Grund dafür ist, dass ChatGPT-4 von OpenAI (2025) leistungsfähig und vielfältig und dabei zwar eingeschränkt, aber kostenlos und öffentlich nutzbar ist. Der Nutzer muss sich lediglich per E-Mail bei OpenAI registrieren und kann ChatGPT-4 von OpenAI (2025) nutzen. Dies jedoch für den kostenlosen Zugang nur eingeschränkt. Die uneingeschränkte Version kostet am 08.02.2025 20 US-Dollar und ist somit ebenfalls kostengünstig. Da ChatGPT-4 von OpenAI (2025) ein weitverbreitetes LLM ist, dass alle Funktionalitäten besitzt und Methoden benutzen kann, welche in Kapitel 2 erläutert werden, bietet es sich gut für diese Arbeit und auch für Entwickler und Unternehmen an. Es kann RAG verwenden und neben Textdokumenten auch Bilder erkennen. Diese Funktion ist für die Testfallgenerierung essenziell, da es sich in dieser Arbeit um Anforderungstestfällen aus Anforderungsspezifikationen handelt. Bei Testdaten, bei welchen es sich um Anwendungen handelt, nach deren Funktionalitäten getestet werden, muss der Tester sich die Anwendung anschauen und aus der Anforderungsspezifikation die Testfälle ableiten. Da ChatGPT-4 von OpenAI (2025) Bilder erkennen und interpretieren kann, kann der Nutzer nun ChatGPT-4 von OpenAI (2025) Screenshots von der Anwendung schicken und diesem mitteilen, um was für eine Art Screenshot es sich handelt, ob es das Startmenü ist oder eine Fehlermeldung. Da ChatGPT-4 von OpenAI (2025) auch eine Erinnerung besitzt, kann es sich diese Informationen leichter merken und bei einem Einspielen der Bilder sich die Struktur auch in anderen Chats merken und nutzen, selbst wenn der ursprüngliche Chat nicht mehr vorhanden ist. Somit ist ChatGPT-4 von OpenAI (2025) kosten günstig verfügbar und besitzt viele Fähigkeiten, welche für die Testfallgenerierung notwendig sind. Es muss somit nicht mit viel Aufwand, Zeit und Ressourcen antrainiert oder ein eigenes Modell entwickelt werden und ist somit universell für alle nutzbar. Dies macht die Erkenntnisse, die aus dieser Arbeit entstehen, wertvoller, weil sie nicht für einen kleinen Kreis, sondern für die Allgemeinheit verwendbar sind.

Der von Müller (2025) dem Autor dieser Arbeit erstellte Screenshot zeigt die Erinnerungsfunktion, welche ChatGPT-4 von OpenAI (2025) besitzt und welche in den Einstellungen zu finden ist.

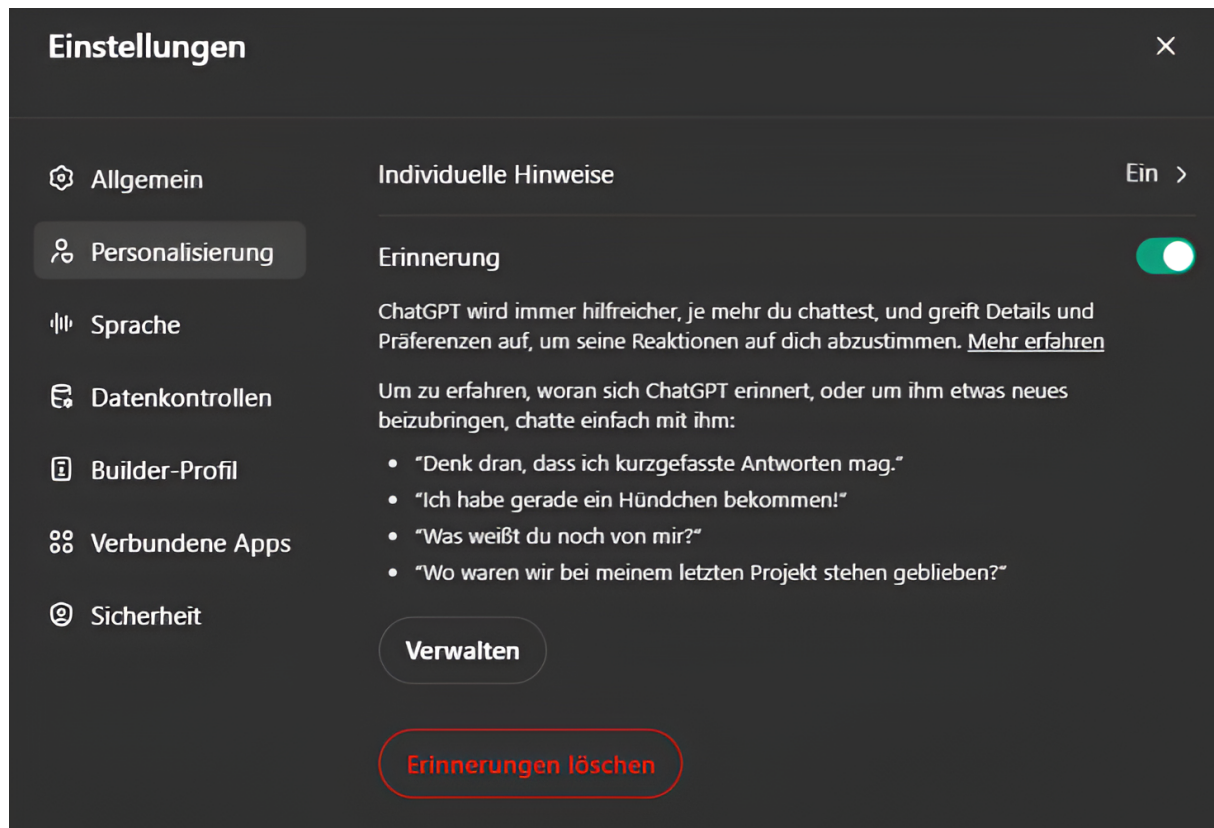


Abbildung 4.1: Ein von Müller (2025) dem Autor dieser Arbeit erstellter Screenshot, welcher die Erinnerungsfunktion von ChatGPT-4 von OpenAI (2025) in den Einstellungen zeigt

Da RAG genutzt werden kann, kann die Anforderungsspezifikation per Textdatei in Form einer PDF-Datei an ChatGPT-4 von OpenAI (2025) übermittelt werden. Mit einem Prompt kann ChatGPT-4 von OpenAI (2025) nun Testfälle erzeugen. Wie die Testdaten aussehen, wie der Prompt aussieht und nach welchen Kriterien die Ergebnisse eingeordnet und ausgewertet werden, wird in Kapitel 4.2, 4.3 und 4.4.4 beschrieben. Ein Vorteil ist auch das ChatGPT-4 von OpenAI (2025) weit verbreitet und ganz leicht im Chatbot mit einer grafischen Benutzeroberfläche genutzt werden kann ohne Programmierkenntnissen, ohne das davor Parameter genau justiert werden müssen und ohne das für diese Arbeit extra ein neues LLM erzeugt oder ein vorhandenes LLM antrainiert werden musste, beweist es das Potenzial für die Testfallgenerierung per KI. Da es leicht zugänglich und leicht ist, gute Testfälle zu erzeugen. ChatGPT-4 von OpenAI (2025) ist auch in sämtlichen Sprachen wie auf Englisch, auf Deutsch, auf Französisch und auf Spanisch nutzbar und kann so von Unternehmen und Softwareentwicklern weltweit genutzt werden. Genauere Informationen, wie ChatGPT-4 von OpenAI (2025) gegenüber anderen Modellen wie zum Beispiel ChatGPT-3.5 von OpenAI (2025) abschneidet, findet sich auf der offiziellen Website von OpenAI (2025). (OpenAI, 2025)

4.2 Testdaten

Als Grundlage zur Erstellung von Testfällen dienen Anforderungsspezifikationen. Um faktische Korrektheit der von ChatGPT-4 von OpenAI (2025) generierten Testfälle messen zu können, ist es essenziell, dass es manuelle Testfälle gibt, welche bereits erstellt wurden. Dadurch können die von ChatGPT-4 von OpenAI (2025) generierten Testfälle in Bezug auf ihre Benutzerfreundlichkeit, Fehlererkennung, Wartbarkeit und Zuverlässigkeit eingeordnet bewertet werden. Für diese Arbeit werden für die Gewinnung von Testdaten, Open Source Projekte von GitHub verwendet, welche Anforderungsspezifikationen und manuelle Testfälle beinhalten. Dabei haben die Autoren (obadaysajjad-SQA-Engineer, 2024; Pavankumar21github, 2022) ebenfalls Open Source Webseiten getestet, sodass sie in dieser Arbeit gezeigt werden können. Der Autor obadaysajjad-SQA-Engineer (2024) testet dabei die Website OrangeHRM (2025). Der Autor Pavankumar21github (2022) testet dabei die Website OpenCart (2025). Folgende Grafik einer Anforderungsspezifikation von Pavankumar21github (2022) zeigt die Anforderungen, welche bei OpenCart (2025) bei der Erstellung eines Accounts erfolgen müssen.

Checkout

Step 1: Checkout Options ▾

New Customer

Checkout Options:

☒ Register Account

☐ Guest Checkout

By creating an account you will be able to shop faster, be up to date on an order's status, and keep track of the orders you have previously made.

Continue

Returning Customer

[I am a returning customer](#)

E-Mail

Password

[Forgot Password](#)

Login

Step 2: Account & Billing Details

Step 3: Delivery Details

Step 4: Delivery Method

Step 5: Payment Method

Step 6: Confirm Order

Step 1 of the check out process allows the user to make an account before continuing with payment. Selecting "Register Account" will change Step 2 of checkout from Billing to Account & Billing details. Account & Billing asks for the same personal details as Billing, except that it asks for the user to create a password for their account. After Step 2 is completed, the customer may continue with the checkout process.

Abbildung 4.2: Eine von Pavankumar21github (2022) erstellte Anforderungsspezifikation, welcher die Anforderungen für die Erstellung eines Accounts in OpenCart (2025) vorgibt

Spezifikationen wie diese werden ChatGPT-4 von OpenAI (2025) zur Verfügung gestellt.

Die generierten Testfälle können nur bewertet werden, wenn es manuelle Testfälle gibt, die die gleichen Funktionen und Anforderungen abdecken. Diese sind ebenfalls in den GitHub-Projekten zu finden. Folgende Grafik zeigt einen Testfall, welcher von Pavankumar21github (2022) erstellt wurde und welcher die Erstellung eines Accounts in OpenCart (2025) testet.

Test Case Title	Pre-requisites	Test Steps			
Validate Registering an Account by providing only the Mandatory fields	1. Open the Application (https://demo.opencart.com) in any Browser	1. Click on 'My Account' Drop menu 2. Click on 'Register' option 3. Enter new Account Details into the Mandatory Fields (First Name, Last Name, E-Mail, Telephone, Password, Password Confirm and Privacy Policy Fields) 4. Click on 'Continue' button (ER-1) 5. Click on 'Continue' button that is displayed in the 'Account Success' page (ER-2)			
Test Data	Expected Result (ER)	Actual Result	Priority	Result	Comments
Not Applicable	1. User should be logged in, taken to 'Account Success' page and proper details should be displayed on the page 2. User should be taken to 'Account' page and a confirm email should be sent to the registered email address	1. User got logged in, taken to 'Account Success' page successfully and proper details got displayed on the page 2. User is taken to 'Account' page and a confirm email received at the registered email address	P1	PASS	

Abbildung 4.3: Ein von Pavankumar21github (2022) erstellter Testfall, welcher die Erstellung eines Accounts in OpenCart (2025) anhand der oben gezeigten Anforderungsspezifikation testet

Mithilfe dieser Anforderungsspezifikation und von Screenshots der jeweiligen Webseite generiert ChatGPT-4 von OpenAI (2025) Testfälle. Wie dieser Vorgang genau abläuft, wird in Kapitel 4.3 erläutert. Diese generierten Testfälle werden in Kapitel 4.4 mithilfe der manuellen Testfälle auf ihre Richtigkeit überprüft und gemessen.

4.3 Prompt und verwendete Methoden

In diesem Kapitel werden das Vorgehen und der Prompt beschrieben, wie in dieser Arbeit Testfälle durch ChatGPT-4 von OpenAI (2025) generiert werden. Zu erst wird wie in Kapitel 4.1 gezeigt, die Erinnerungsfunktion aktiviert, damit sich ChatGPT-4 von OpenAI (2025) an die Informationen erinnern kann, welche ihm mitgegeben werden. Anschließend werden ChatGPT-4 von OpenAI (2025) Screenshots von der jeweiligen Website übergeben, da wie in Kapitel 4.1 beschrieben ChatGPT-4 von OpenAI (2025) Bilder erkennen und einordnen kann, erhält es somit die Informationen, was auf der Website passiert und welche Funktionen oder Seiten geöffnet werden, wenn eine Anforderung ausgeführt wird. Dadurch erhält es die Informationen darüber, wie die Seite richtig funktioniert und kann somit diese im positiven und im negativen Fall testen. Danach wird ChatGPT-4 von OpenAI (2025) die jeweilige Anforderungsspezifikation für die Testfälle als PDF übergeben. Die Anforderungsspezifikation sieht dabei so aus, wie sie in Kapitel 4.2 beschrieben werden. Durch die in Kapitel 2.4 beschriebene Methode RAG kann es dadurch die Informationen besser bearbeiten.

Anschließend wird ein Prompt übergeben welcher, in folgender Grafik gezeigt wird.

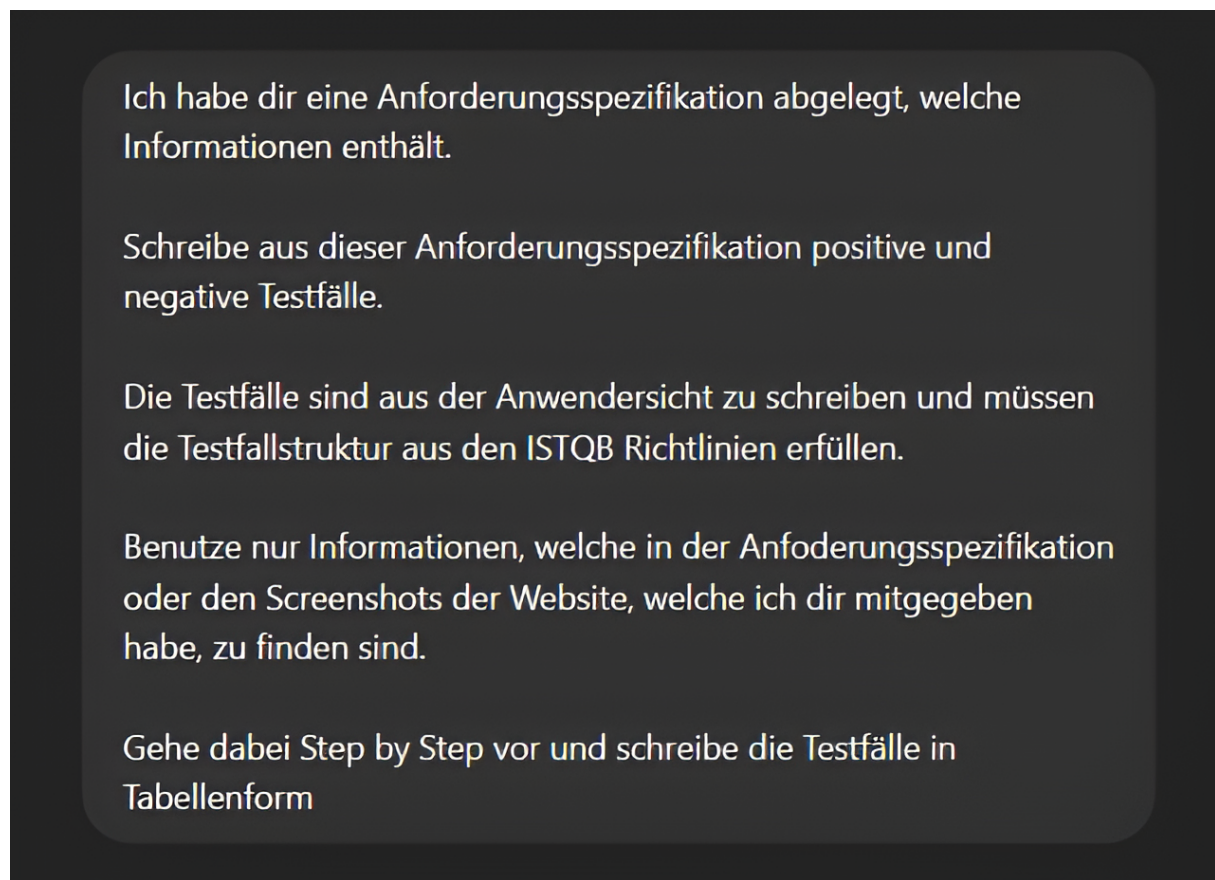


Abbildung 4.4: Ein von Müller (2025), dem Autor dieser Arbeit erstellter Screenshot, welcher den Prompt an ChatGPT-4 von OpenAI (2025) zeigt

Der Prompt ist dabei so geschrieben, dass ChatGPT-4 von OpenAI (2025) Schritt für Schritt weis, welche Aufgaben es erfüllen soll. Er soll dabei die Informationen aus der Anforderungsspezifikation und aus den Screenshots benutzen und dabei die Struktur aus ISO/IEC/IEEE 29119-1:2022-01 (2022) erfüllen. Dies ist für Kapitel 4.4.2 wichtig, da somit die Ausschlusskriterien bereits im Prompt definiert sind und ChatGPT-4 von OpenAI (2025) diese erfüllen soll.

Nach dieser Eingabe erstellt ChatGPT-4 von OpenAI (2025) die jeweiligen Testfälle pro Anforderung. Diese Testfälle werden per Screenshot gespeichert und sind im GitHub-Projekt von Müller (2025) gespeichert. Anschließend werden diese mit den manuell erstellten Testfällen, wie in Kapitel 4.2 erläutert, auf ihre Korrektheit überprüft. Anschließend werden diese ausgewertet. Die Metriken zu der Auswertung der generierten Testfälle findet sich in Kapitel 4.4.

4.4 Auswertungsmetriken

Im Bereich des Testens gibt es viele Metriken und Qualitätskriterien, anhand derer gemessen werden kann, wie gut ein Testfall ist. Dabei spielt es für die Auswertung keine Rolle, ob der Testfall manuell oder mithilfe einer LLM wie ChatGPT-4 von OpenAI (2025) erstellt wird. Anhand dieser hier dargestellten Ausschluss- und Qualitätskriterien werden die von ChatGPT-4 von OpenAI (2025) erstellten Testfälle ausgewertet und nach ihrer Qualität gemessen. In dieser Arbeit gibt es zwei Arten von Metriken, zum einen die Ausschlusskriterien, Kriterien, welche die von ChatGPT-4 von OpenAI (2025) generierten Testfälle erfüllen müssen, um bewertet zu werden und zum anderen die Qualitätskriterien, anhand derer diese mit manuellen Testfällen, die die gleichen Anforderungen abdecken, bewertet werden.

4.4.1 Eigenschaften guter Testfälle

Im Bereich der Softwareentwicklung und der Testfälle gibt es unzählige Kriterien und Metriken, anhand derer guter Code und gute Testfälle ausgewertet werden können. Wie in Kapitel 3 geschrieben steht, sind die Werke von (Lazić, 2013; Nirpals & Kale, 2011) zu nennen.

Da in dieser Arbeit wenig Mittel und Ressourcen zur Verfügung stehen, die von ChatGPT-4 von OpenAI (2025) generierten Testfälle nach all diesen Kriterien und Metriken auszuwerten, ist es wichtig, wichtige Kriterien und Metriken zu finden. In Kapitel 3 steht, dass die Autoren Tran et al. (2025) zu der Erkenntnis kommen, dass die vier Eigenschaften, Fault Detection (Deutsch: Fehlererkennung), Maintainability (Deutsch: Wartbarkeit), Reliability (Deutsch: Zuverlässigkeit) und Usability (Deutsch: Benutzerfreundlichkeit) im Bereich Testfall die wichtigsten Kriterien sind.

Dies haben sie durch eine Befragung mit vielen Personen, die im Bereich des Testing herausgefunden. Dabei haben sie eine Umfrage mit ihnen gemacht und diese Personen nach den wichtigen Eigenschaften im Bereich des Testing gefragt und deren Antworten ausgewertet. In dieser Arbeit wird mithilfe der Ausschlusskriterien und der Qualitätskriterien die durch ChatGPT-4 von OpenAI (2025) generierten Testfälle ausgewertet, wie sehr diese die Eigenschaften erfüllen.

Die Grafik von Tran et al. (2025) zeigt dabei, wie die einzelnen Qualitätskriterien abschneiden:

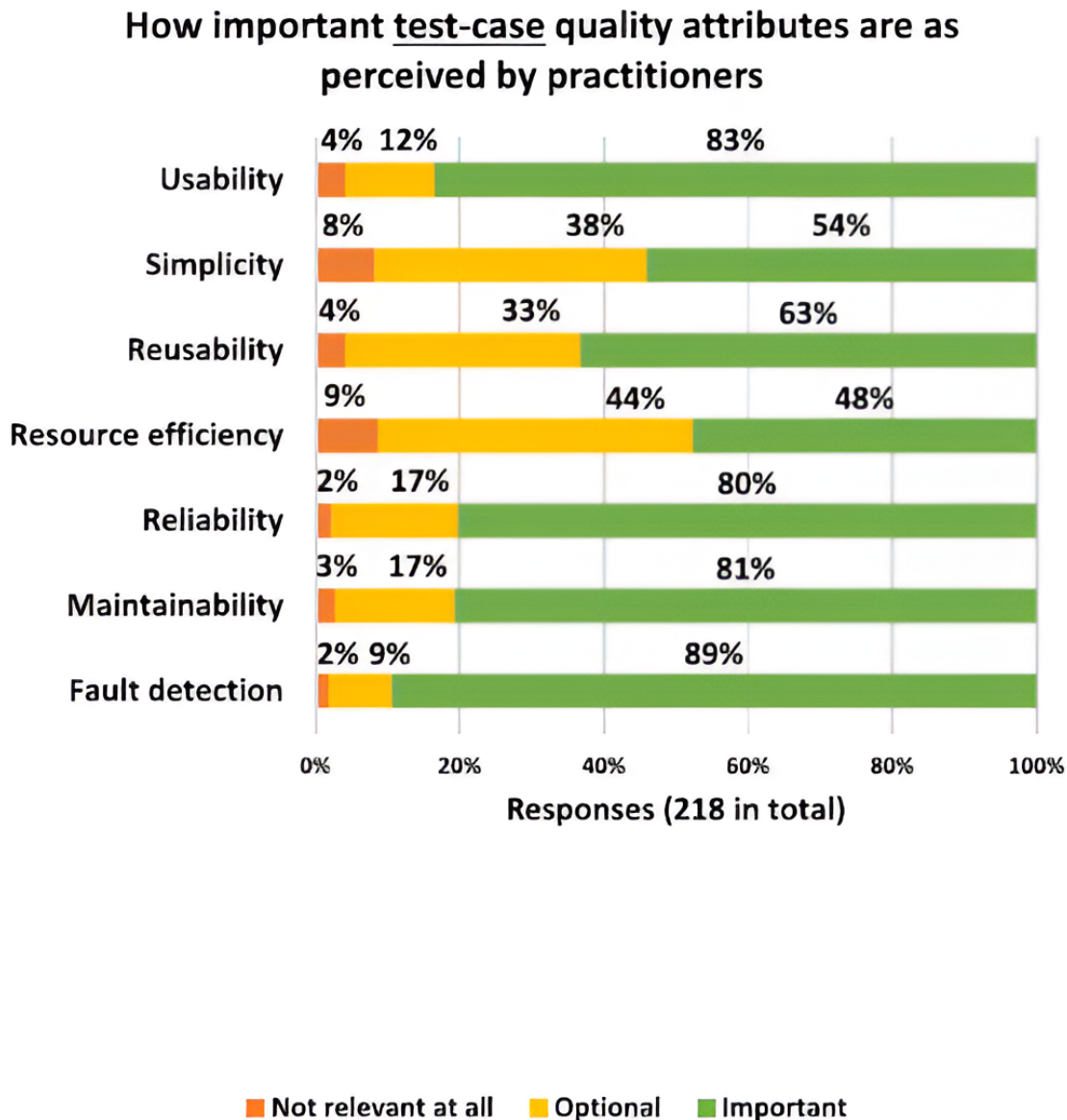


Abbildung 4.5: Die Grafik von Tran et al. (2025) zeigt, wie die einzelnen Qualitätskriterien von den Teilnehmern ihrer Befragung in ihrer Wichtigkeit erachtet werden

Diese vier Eigenschaften sind Eigenschaften, die sowohl in der Softwareentwicklung als auch im Bereich des Testing wichtig sind. In Kapitel 4.4.2 und in Kapitel 4.4.4 werden Metriken aufgestellt, anhand derer die generierten Testfälle gemessen werden können. Dadurch kann in Kapitel 5 bewertet werden, ob die durch ChatGPT-4 von OpenAI (2025) generierten Testfälle diese Eigenschaften erfüllen.

4.4.2 Ausschlusskriterien

In dieser Arbeit gibt es zwei Ausschlusskriterien, die Testfallstruktur und die Kohärenzrate (KR). Die von ChatGPT-4 von OpenAI (2025) generierten Testfälle müssen beide erfüllen, damit sie mit den Qualitätskriterien bewertet werden können.

1.1 Testfallstruktur:

In Kapitel 2.5 wird die Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) beschrieben. Ein Testfall nach ISO/IEC/IEEE 29119-1:2022-01 (2022) besitzt folgende Merkmale: Ein Identifier, einen Namen, eine Beschreibung, Vorbedingungen und Ziele. Dabei testet der Testfall einen Ablauf, welcher sich in Schritten gliedert. In dieser Arbeit wird von ChatGPT-4 von OpenAI (2025) erwartet, dass es, wenn es einen Testfall generiert, es diesen Aufbau einhält. Der Grund ist folgender, für viele Entwickler und Unternehmen ist es wichtig, dass Standards eingehalten werden. Ein Testfall, welcher vom Aufbau her diesen Standard nicht erfüllt, ist nicht brauchbar und muss somit auch negativ bewertet werden. Folgendes Beispiel zeigt einen Testfall, welcher die Anforderungen aus der in Kapitel 4.2 gezeigten Anforderungsspezifikation testen soll, welcher nicht die Testfallstruktur erfüllt und somit nicht weiter berücksichtigt wird, da er unbrauchbar ist.

Testfall	Ein Benutzer erstellt ein Konto während des Checkouts.
Schritte	1. Gehe zur Seite. 2. Wähle Register. 3. Weiter drücken. 4. Daten eingeben. 5. Weiter.
Erwartetes Ergebnis	Sollte funktionieren.

Abbildung 4.6: Ein von ChatGPT-4 von OpenAI (2025) erstellter Testfall, welcher nicht der Testfallstruktur entspricht

Bei der Auswertung wird überprüft und gemessen, ob ChatGPT-4 von OpenAI (2025) standardmäßig die Testfallstruktur von ISO/IEC/IEEE 29119-1:2022-01 (2022) benutzt. Das Kriterium Testfallstruktur ist das Ausschlusskriterium für diese Arbeit und die Auswertung der Testfälle, erst wenn die generierten Testfälle diesen Aufbau besitzen, können diese für weitere Qualitätskriterien ausgewertet werden. Deshalb wird überprüft und gemessen, wie viele der von ChatGPT-4 von OpenAI (2025) generierten Testfälle diese Testfallstruktur ganz oder teilweise besitzen.

Dieser Wert hat zwei Kategorien, welche in folgender Tabelle abgebildet sind:

Testfallstruktur:

	1. Positiv	2. Negativ
Einstufung	0 Merkmale fehlen	$1 \geq \text{Merkmale fehlen}$

Am Ende wird angegeben, wie viel Prozent der Testfälle in der positiven oder in der negativen Kategorie eingeordnet werden. Testfälle welche sich in der 2. Kategorie befinden, werden nicht weiter ausgewertet, da diese nicht brauchbar sind.

Problem der Kohärenz

Die Testfallstruktur ist wichtig, jedoch nicht der einzige wichtige Punkt für die Anforderung an einen Testfall. Die Logik und der Zusammenhang müssen ebenfalls stimmen. Die Testschritte müssen logisch aufgebaut sein und in der richtigen Reihenfolge stehen. So kann ein Nutzer sich nicht von einer Anwendung abmelden, wenn er sich nicht vorher angemeldet hat. Ein Testfall muss somit als ersten Testschritt Anmeldung besitzen, bevor dieser den Testschritt Abmeldung hat. Folgendes Beispiel zeigt einen von ChatGPT-4 von OpenAI (2025) generierten Testfall, welcher zwar die richtige Teststruktur besitzt und somit das erste Ausschlusskriterium erfüllt, aber die Testschritte nicht logisch und nicht in der richtigen Reihenfolge sind.

Testfall-ID	TC-002
Testfall-Name	Registrierung eines neuen Kunden im Checkout-Prozess
Beschreibung	Dieser Testfall überprüft, ob ein neuer Kunde erfolgreich ein Konto erstellen und den Checkout-Prozess fortsetzen kann.
Voraussetzungen	- Der Benutzer hat Zugriff auf die Checkout-Seite. - Der Benutzer hat noch kein Konto.
Testschritte	1. Überprüfe, ob der Benutzer erfolgreich zum nächsten Schritt des Checkouts weitergeleitet wird. 2. Klicke auf „Continue“. 3. Fülle alle erforderlichen Felder im Abschnitt „Account & Billing Details“ aus (Name, Adresse, E-Mail, Passwort). 4. Wähle die Option „Register Account“. 5. Öffne die Checkout-Seite. 6. Klicke auf „Continue“.
Erwartetes Ergebnis	Der Benutzer wird erfolgreich registriert und kann mit dem Checkout-Prozess fortfahren.

Abbildung 4.7: Ein von ChatGPT-4 von OpenAI (2025) erstellter Testfall, welcher nicht kohärente Testschritte besitzt.

Der erste Testschritt sollte „Öffne die Checkout-Seite“ sein, dieser Testschritt ist jedoch in diesem Beispiel der fünfte Testschritt.

Deshalb ist dieser Testfall unbrauchbar, da dieser nicht die Logik der Anforderungsspezifikation erfüllt. Ein weiteres Problem für die Kohärenz ist das Problem der Halluzination. In Kapitel 1.1 steht, dass Siebert (2024) den Begriff Halluzination als Phänomen beschreibt, bei dem eine KI Antwort nicht mit den gegebenen Informationen übereinstimmt und sich Antworten ausdenkt, die faktisch nicht richtig sind. Da in dieser Arbeit die Testfälle von ChatGPT-4 von OpenAI (2025) generiert werden, ist ein weiteres Kriterium für deren Auswertung wichtig. Die Autoren Li et al. (2024) stellen eine Metrik auf, um die Anzahl der Halluzinationen zu messen, die Halluzinationsrate (HR). Diese misst die Anzahl der Halluzinationen im Verhältnis zu der Gesamtantwort. In dieser Arbeit wird diese Metrik für den Anwendungszweck der Testfälle modifiziert verwendet, die sogenannte Kohärenzrate (KR).

1.2 Kohärenzrate (KR):

Mit dem Qualitätskriterium Kohärenzrate (KR) wird in dieser Arbeit an Li et al. (2024) angelehnt, das Verhältnis der Anzahl der kohärenten Testschritte pro Testfall und der Gesamtzahl an Testschritten des Testfalls gemessen. Ein kohärenter Testschritt steht in der richtigen logischen Reihenfolge, ist keine Halluzination und besitzt seine Informationen aus der Anforderungsspezifikation oder dem Screenshot aus der Webseite. Statt wie Li et al. (2024) den negativen Anteil zu messen, wird mit dieser Vorgehensweise der positive Anteil gemessen.

$$KR = \frac{KT}{T} \times 100\%$$

Dabei sind:

- KT Anzahl der kohärenten Testschritte pro Testfall,
- T die Gesamtanzahl der generierten Testschritte pro Testfall.

Durch diese Metrik kann der Anteil der kohärenten Testschritte pro Testfall in Prozent % gemessen werden. Um die Kohärenz der Testschritt zu bestimmen, werden die von ChatGPT-4 von OpenAI (2025) generierten Testfälle mit den vorhandenen manuellen Testfällen verglichen. Ein Testschritt ist nicht kohärent, wenn in einem Testschritt falsche Begriffe oder Schritte vorkommen oder dieser Funktionen testet, die weder in der Anforderungsspezifikation, dem manuellen Testfall oder auf der Website zu finden sind. Ein Testschritt, welcher in der falschen logischen Reihenfolge steht, ist ebenfalls nicht kohärent. Die Kohärenzrate hat zwei Kategorien, welche in folgender Tabelle abgebildet sind.

Kohärenzrate:

	1. Positiv	2. Negativ
Anteil KT in %	100 %	100 % <

Es wird eingeordnet wie viel Prozent % der Testfälle, in welcher der zwei Kategorien eingeordnet werden. Anschließend wird der Mittelwert berechnet, um die Gesamtrate aller Testfälle zu ermitteln. Von einem Testfall wird erwartet, dass er sich in der 1. Kategorie befindet.

Folgendes Beispiel zeigt einen von ChatGPT-4 von OpenAI (2025) erstellten Testfall, welcher sowohl die richtige Testfallstruktur als auch nur kohärente Testschritte aufweist.

Testfall-ID	TC-001
Testfall-Name	Registrierung eines neuen Kunden im Checkout-Prozess
Beschreibung	Dieser Testfall überprüft, ob ein neuer Kunde erfolgreich ein Konto erstellen und den Checkout-Prozess fortsetzen kann.
Voraussetzungen	- Der Benutzer hat Zugriff auf die Checkout-Seite. - Der Benutzer hat noch kein Konto.
Testschritte	1. Öffne die Checkout-Seite. 2. Wähle die Option „Register Account“. 3. Klicke auf „Continue“. 4. Fülle alle erforderlichen Felder im Abschnitt „Account & Billing Details“ aus (Name, Adresse, E-Mail, Passwort). 5. Klicke auf „Continue“. 6. Überprüfe, ob der Benutzer erfolgreich zum nächsten Schritt des Checkouts weitergeleitet wird.
Erwartetes Ergebnis	Der Benutzer wird erfolgreich registriert und kann mit dem Checkout-Prozess fortfahren.

Abbildung 4.8: Ein von ChatGPT-4 von OpenAI (2025) erstellter Testfall, welcher der Testfallstruktur entspricht und kohärente Testschritte besitzt

Nur wenn ein Testfall die richtige Teststruktur und kohärente Testschritte besitzt, ist es sinnvoll, sie auf ihre Qualität in Bezug auf Benutzerfreundlichkeit, Fehlererkennung, Wartbarkeit und Zuverlässigkeit auszuwerten.

4.4.3 Qualitätsmetriken

In dieser Arbeit gibt es vier Qualitätsmetriken nach Wang und Zhu (2024). Diese werden für diese Arbeit modifiziert verwendet, um die Qualität von durch ChatGPT-4 von OpenAI (2025) generierten Testfälle, welche die Ausschlusskriterien erfüllen, nach Tran et al. (2025) zu bewerten. Für die Bestimmung der vier Qualitätsmetriken nach Wang und Zhu (2024) werden die vier Werte definiert:

- **True Positive (TP):** Ein falscher Testschritt wurde korrekt als falsch erkannt.
- **False Positive (FP):** Ein richtiger Testschritt wurde als falsch eingeordnet.
- **True Negative (TN):** Ein richtiger Testschritt wurde als richtig erkannt.
- **False Negative (FN):** Ein falscher Testschritt wurde als richtig eingeordnet.

Diese Werte werden ermittelt, in dem ChatGPT-4 von OpenAI (2025) dazu aufgefordert wird, einen positiven und einen negativen Testfall und somit ein Testfallpaar zu schreiben. In dem positiven Testfall werden TN und FN und in dem negativen Testfall werden TP und FP ermittelt. Das Testfallpaar wird dann zusammen ausgewertet.

2.1 Accuracy (Deutsch: Genauigkeit)

Die Genauigkeit nach Wang und Zhu (2024) misst den Anteil der insgesamt richtig eingeordneten Testschritte:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

Die Anzahl der Testschritte misst sich dabei, indem die Anzahl der Testschritte des positiven und den mit ihm zusammenhängenden negativen Testfalls addiert werden. Die Werte werden in Kapitel 4.4.3 für TP, FP, TN und FN definiert. Es wird die Genauigkeit aller Testfälle als Mittelwert angegeben und ein Boxplot erstellt, um die Streuung der Genauigkeit darzustellen.

2.2 Precision (Deutsch: Präzision)

Die Präzision nach Wang und Zhu (2024) ermittelt, wie viele der als falsch eingestufenen Testschritte wirklich falsch sind:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Die Präzision des Testfallpaares wird aus dem Ergebnis des negativen Testfalls ermittelt. In Kapitel 4.4.3 werden TP und FP definiert. Für die negativen Testfälle wird der Mittelwert der Präzision berechnet und ein Boxplot erstellt, um deren Streuung zu zeigen.

2.3 Recall (Deutsch: Sensitivität)

Die Sensitivität nach Wang und Zhu (2024) zeigt, wie viele der falschen Testschritte als falsch erkannt wurden:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Die Sensitivität des Testfallpaares wird aus dem Ergebnis beider Testfälle gemessen. TP und FN werden in Kapitel 4.4.3 definiert. Um die Streuung der Sensitivität zu veranschaulichen, wird ein Boxplot aller Werte erstellt. Ebenso wird der Mittelwert der Sensitivität aus allen Testfallpaaren ermittelt.

2.4 False Positive Rate (Deutsch: Falsch-Positiv-Rate)

Die Falsch-Positiv-Rate nach Wang und Zhu (2024) gibt an, wie viele der als falsch gewerteten Testschritte richtig waren:

$$\text{False Positive Rate} = \frac{FP}{FP + TN}$$

Die Falsch-Positiv-Rate des Testfallpaares misst sich dabei aus dem Ergebnis beider Testfälle. Die Werte für FP und TN werden in Kapitel 4.4.3 definiert. Um diese Metrik für die Auswertung der in 4.4.4 definierten Qualitätskriterien nutzen zu können, wird der Mittelwert der Falsch-Positiv-Rate aus allen Testfallpaaren bestimmt und um die Streuung der Falsch-Positiv-Rate zu veranschaulichen, ein Boxplot erstellt.

4.4.4 Qualitätskriterien

In dieser Arbeit gibt es vier Qualitätskriterien, die von Tran et al. (2025) ermittelt wurden. Diese werden verwendet, um die von ChatGPT-4 von OpenAI (2025) generierten Testfälle auszuwerten. Dabei helfen die in Kapitel 4.4.2 und in Kapitel 4.4.3 aufgestellten Kriterien und Metriken. Wie in Kapitel 4.4.1 geschrieben, stellen Tran et al. (2025) Qualitätskriterien vor, anhand derer gute Testfälle erkannt werden können. Dabei definieren Tran et al. (2025) diese folgend und stellen Anforderungen auf, um diese zu messen.

1. Fault Detection (Deutsch: Fehlererkennung)

Den Begriff Fehlererkennung definieren Tran et al. (2025) als ein Kriterium, welches die Effektivität und Effizienz misst, in welchem Maß Testfälle Fehler erkennen können. Dabei zeichnet sich nach Tran et al. (2025) eine effiziente Fehlererkennung bei einem Testfall dadurch aus, dass dieser Fehler schnell findet. Wie viele Fehler dieser findet, ist ebenfalls wichtig. Die in Kapitel 4.4.3 Metrik nach Wang und Zhu (2024) Sensitivität eignet sich für das Messen einer guten Fehlererkennung. Dieser misst, wie viele falsche Testschritte als falsch erkannt werden. Die Höhe der Sensitivität misst somit wie gut durch ChatGPT-4 von OpenAI (2025) generierten Fehler erkennen können.

2. Maintainability (Deutsch: Wartbarkeit)

Ein Kriterium, welches nach Tran et al. (2025) einen Testfall daran misst, wie dieser sich in Bezug auf Fehlerkorrektur, Verbesserungen und Änderungen in den Anforderungen anpassen kann, ist die Wartbarkeit. Dabei spielen laut Tran et al. (2025) die Analysefähigkeit, wie gut sie Mängel und Fehler identifizieren können, die Nachverfolgbarkeit, wie sehr der Testfall mit den Anforderungen verbunden ist und die Homogenität, die kontinuierliche Einhaltung der gleichen Struktur und Regeln, eine Rolle.

Die in Kapitel 4.4.2 stehende Ausschlusskategorie Testfallstruktur, welche sicherstellt, dass die Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) eingehalten wird, eignet sich gut, um die Homogenität zu messen. Es wird so sichergestellt, dass die von ChatGPT-4 von OpenAI (2025) erstellten Testfälle nach der gleichen Struktur und Regeln erstellt werden.

Die Kohärenzrate, welche in Kapitel 4.4.2 aufgestellt wird und welche an Li et al. (2024) angelehnt ist, stellt sicher, dass die von ChatGPT-4 von OpenAI (2025) erstellten Testfälle alle Schritte aus den Anforderungen einhalten. Damit kann durch diesen Wert die Nachverfolgbarkeit bewertet werden und stellt dabei sicher, dass es keine unlogischen Testschritte gibt, die später gewartet werden müssen.

Die in Kapitel 4.4.3 nach Wang und Zhu (2024) modifizierte Metrik Falsch-Positiv-Rate gibt an, wie viele als falsch eingestufte Testschritte in Wahrheit richtig sind. Eine niedrige Falsch-Positiv-Rate zeigt, dass es kaum falsche Alarmer gibt, welche mit einem unnötigen Wartungsaufwand verbunden sind, da sie als vermeintliche Fehler korrigiert werden müssen, obwohl sie richtig sind.

3. Reliability (Deutsch: Zuverlässigkeit)

Die Kriterien Fehlererkennung und Wartbarkeit sind wichtig, bringen jedoch nicht viel, wenn die von ChatGPT-4 von OpenAI (2025) generierten Testfälle eine gute Fehlererkennung und Wartbarkeit nicht dauerhaft und in verschiedenen Bereichen aufrechterhalten können und somit nicht zuverlässig sind. Das Kriterium Zuverlässigkeit wird deshalb von Tran et al. (2025) als eine Fähigkeit beschrieben, trotz unterschiedlicher Anforderungen und Bedingungen ein Leistungsniveau aufrechtzuerhalten.

In dieser Arbeit werden von ChatGPT-4 von OpenAI (2025) Testfälle mit verschiedenen Anforderungen und verschiedene Websites getestet. Wenn die in Kapitel 4.4.2 aufgestellte Kohärenzrate nach Li et al. (2024) und die in Kapitel 4.4.3 beschriebene Metriken Genauigkeit von Wang und Zhu (2024), welche die Anzahl der richtig eingeordneten Testschritte misst und Präzession, welche misst wie viele als falsch eingestuften Testschritte tatsächlich falsch sind, im Mittelwert hoch sind, zeigt es, dass die von ChatGPT-4 von OpenAI (2025) erstellten Testfälle über unterschiedliche Anforderungen hinweg logisch und richtig sind.

4. Usability (Deutsch: Benutzerfreundlichkeit)

Benutzerfreundlichkeit misst nach Tran et al. (2025) Testfälle daran, wie leicht diese ausgeführt, verwendet und verstanden werden können. Dabei spielen laut Tran et al. (2025) Vollständigkeit, Lesbarkeit und Verständlichkeit eine Rolle. Das in Kapitel 4.4.2 aufgestellte Kriterium Testfallstruktur stellt sicher, dass die durch ChatGPT-4 von OpenAI (2025) erstellten Testfälle die Testfallstruktur nach ISO/IEC/IEEE 29119-1:2022-01 (2022) erfüllen. Dieses Kriterium stellt die Benutzerfreundlichkeit der Testfälle sicher, falls diese eingehalten wird, da sie dann übersichtlich und nach einem Standard aufgebaut sind.

Anhand dieser vier von Tran et al. (2025) aufgestellten Qualitätskriterien werden die durch ChatGPT-4 von OpenAI (2025) erstellten Testfälle in Kapitel 5 ausgewertet und diskutiert. In Kapitel 6 werden diese Ergebnisse eingeordnet.

5 Ergebnisse

In Kapitel 5 werden die Ergebnisse der von der KI erstellten Anforderungstestfälle dokumentiert, beschrieben und eingeordnet. Alle Testdaten, die Anforderungsspezifikationen, die manuellen sowie die von ChatGPT-4 von OpenAI (2025) generierten Testfälle und die Auswertungstabellen finden sich im GitHub-Projekt von Müller (2025) dem Autor dieser Arbeit.

5.1 Vorliegende Ergebnisse

Dies sind die Ergebnisse der KI TODO Ergebnisse zeigen

1.1 Testfallstruktur:

1.2 Kohärenzrate (KR)

2.1 Fault Detection (Deutsch: Fehlererkennung)

2.2 Maintainability (Deutsch: Wartbarkeit)

2.3 Reliability (Deutsch: Zuverlässigkeit)

2.4 Usability (Deutsch: Benutzerfreundlichkeit)

5.2 Auswertung der Ergebnisse

Die von der KI generierten Testfälle schneiden wie folgt ab: TODO Ergebnisse auswerten und beschreiben

1.1 Testfallstruktur:

1.2 Kohärenzrate (KR)

2.1 Fault Detection (Deutsch: Fehlererkennung)

2.2 Maintainability (Deutsch: Wartbarkeit)

2.3 Reliability (Deutsch: Zuverlässigkeit)

2.4 Usability (Deutsch: Benutzerfreundlichkeit)

5.3 Diskussion

Der Grund warum die Testfälle so abschneiden wie sie abschneiden, kann an folgenden Punkten liegen: TODO Nach der Auswertung Ergebnisse einordnen und Grund dafür

nennen

6 Fazit und Ausblick

Über die Einordnung der Ergebnisse dieser Arbeit und mögliche Bedeutungen für die Zukunft wird in Kapitel 6 Fazit und Ausblick Auskunft gegeben.

Literatur

- Aysolmaz, B., Leopold, H., Reijers, H. A., & Demirörs, O. (2018). A semi-automated approach for generating natural language requirements documents based on business process models. *Information and Software Technology*, 93, 14–29. <https://www.sciencedirect.com/science/article/abs/pii/S0950584917302069?via%3Dihub>
- Barmi, Z. A., Ebrahimi, A. H., & Feldt, R. (2011). Alignment of Requirements Specification and Testing: A Systematic Mapping Study. *2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*. <https://ieeexplore.ieee.org/document/5954452>
- Bhandari, J., Knechtel, J., Narayanaswamy, R., Garg, S., & Karri, R. (2024). LLM-Aided Testbench Generation and Bug Detection for Finite-State Machines. *Computer Science > Hardware Architecture*, 1. <https://arxiv.org/abs/2406.17132>
- Bhatt, C., Kumar, I., Vijayakumar, V., Singh, K. U., & Kumar, A. (2021). The state of the art of deep learning models in medical science and their challenges. *Multimedia Systems*, 4, 599–613. <https://link.springer.com/article/10.1007/s00530-020-00694-1>
- Bozic, S. (2022). *Künstliche Intelligenz im Testprozess* [abgerufen am 03.11.2024]. <https://bbv-software.de/insights/blog/ai-testing/>
- Chang, Y., Wang, X., Wang, J., Wu, Y., Yang, L., Zhu, K., Chen, H., Yi, X., Wang, C., Wang, Y., Ye, W., Zhang, Y., Chang, Y., Yu, P. S., Yang, Q., & Xie, X. (2023). A Survey on Evaluation of Large Language Models. *Computer Science > Computation and Language*, 9. <https://arxiv.org/abs/2307.03109>
- Dymatrix. (2018). *Deep-Learning im Digital Business* [abgerufen am 13.01.2025]. <https://www.dymatrix.de/dymatrixblog/deep-learning-definition>
- Enderli, D. (2019). *Testfalldesign für SAP Applikationen* [abgerufen am 16.01.2025]. <https://community.sap.com/t5/technology-blogs-by-members/testfalldesign-f%C3%BCr-sap-applikationen/ba-p/13451295>
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2024). Retrieval-Augmented Generation for Large Language Models: A Survey. *Computer Science > Computation and Language*, 5. <https://arxiv.org/abs/2312.10997>
- Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd Edition* [ISBN: 978-1-4920-3264-9]. O'Reilly Media, Inc.
- Gu, J., Han, Z., Chen, S., Beirami, A., He, B., Zhang, G., Liao, R., Qin, Y., Tresp, V., & Torr, P. (2023). A Systematic Survey of Prompt Engineering on Vision-Language Foundation Models. *Computer Science > Computer Vision and Pattern Recognition*, 1. <https://arxiv.org/abs/2307.12980>
- Gu, S., Fang, C., Zhang, Q., Tian, F., Zhou, J., & Chen, Z. (2024). TestART: Improving LLM-based Unit Test via Co-evolution of Automated Generation and Repair Iteration. *Computer Science > Software Engineering*, 5. <https://arxiv.org/abs/2408.03095>
- Hecker, D., Döbel, I., Petersen, U., Rauschert, A., Schmitz, V., & Voss, A. (2018). Zukunftsmarkt Künstliche Intelligenz. <https://kmu-digital.eu/de/service-kompetenz/publikationen/dokumente/studien/198-zukunftsmarkt-kuenstliche-intelligenz-potenzielle-und-anwendungen>

- Hodiak, V. (2024). *NLP vs LLM: Navigating Differences and Core Advantages* [abgerufen am 16.01.2025]. <https://otakoyi.software/blog/nlp-vs-llm-navigating-differences-and-core-advantages>
- Honroth, T., Siebert, J., & Kelbert, P. (2024). *Retrieval Augmented Generation (RAG): Chatten mit den eigenen Daten* [abgerufen am 13.01.2025]. <https://www.iese.fraunhofer.de/blog/retrieval-augmented-generation-rag/>
- Hugging Face. (2025). *The AI community building the future*. [abgerufen am 16.01.2025]. <https://huggingface.co/>
- Iqbal, T., & Qureshi, S. (2022). The survey: Text generation models in deep learning. *Journal of King Saud University – Computer and Information Sciences*, 34(6), 2515–2528. <https://doi.org/10.1016/j.jksuci.2020.04.001>
- ISO/IEC/IEEE 29119-1:2022-01. (2022). *Software and systems engineering — Software testing Part 1: General concepts* [abgerufen am 16.01.2025]. <https://www.iso.org/standard/81291.html>
- ISTQB/GTB Standardglossar der Testbegriffe. (2017). *Standardglossar der Testbegriffe* [abgerufen am 16.01.2025]. https://swisstestingboard.org/wp-content/uploads/2018/Documents/Glossar/Glossar%20Deutsch-Englisch%203_11.pdf
- Jama Software. (2024). *Functional vs. nonfunctional requirements: What's the difference?* [abgerufen am 16.01.2025]. <https://www.jamasoftware.com/requirements-management-guide/writing-requirements/functional-vs-non-functional-requirements>
- Kaddour, J., Harris, J., Mozes, M., Bradley, H., Raileanu, R., & McHardy, R. (2023). Challenges and Applications of Large Language Models. *Computer Science > Computation and Language*, 1. <https://arxiv.org/abs/2307.10169>
- Kerner, S. M. (2024). *Large Language Model (LLM)* [abgerufen am 03.11.2024]. <https://www.computerweekly.com/de/definition/Large-Language-Model-LLM>
- Khan, F. I., Mahmud, F. U., & Hoseen, A. (2024). A New Approach of Software Test Automation Using AI. *Journal of Basic Science and Engineering*, 21(2), 559–570. https://www.researchgate.net/publication/380459206_A_NEW_APPROACH_OF_SOFTWARE_TEST_AUTOMATION_USING_AI
- Lazić, L. (2013). Software Quality Testing Metrics. *Computer Science*. <https://www.semanticscholar.org/paper/Software-Quality-%26-Testing-Metrics-Lazic/2df2f394184e3620e7471>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2021). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Computer Science > Computation and Language*, 4. <https://arxiv.org/abs/2005.11401>
- Li, J., Chen, J., Ren, R., Cheng, X., Zhao, W. X., Nie, J.-Y., & Wen, J.-R. (2024). The Dawn After the Dark: An Empirical Study on Factuality Hallucination in Large Language Models. *Computer Science > Computation and Language*, 1. <https://arxiv.org/abs/2401.03205>
- Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., & Neubig, G. (2023). Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *ACM Computing Surveys, Volume 55, Issue 9, 1*, 1–35. <https://dl.acm.org/doi/10.1145/3560815>
- Liu, Z., Chen, C., Wang, J., Chen, M., Wu, B., Che, X., Wang, D., & Wang, Q. (2024). Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions. *ICSE '24: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 46(100), 1–13. <https://dl.acm.org/doi/10.1145/3597503.3639180>

- Miesle, P. (2023). *Retrieval-Augmented Generation (RAG) Explained: Understanding Key Concepts* [abgerufen am 13.01.2025]. <https://www.datastax.com/guides/what-is-retrieval-augmented-generation>
- Mocko, M., & Bitton-Bailey, A. (2021). *Areas of Artificial Intelligence* [abgerufen am 13.01.2025]. <https://ufl.pb.unizin.org/artificialintelligence/chapter/six-area-of-artificial-intelligence/>
- Möllers, N. (2024). *Künstliche Intelligenz und Datenschutz* [abgerufen am 03.11.2024]. <https://keyed.de/blog/kuenstliche-intelligenz-und-datenschutz/>
- Müller, M. (2025). *MrMueller15/Bachelor-Thesis* [abgerufen am 11.02.2025]. <https://github.com/MrMueller15/Bachelor-Thesis>
- Mustafa, A., Wan-Kadir, W. M. N., Ibrahim, N., Shah, M. A., Younas, M., Khan, A., Zareei, M., & Alanazi, F. (2021). Automated Test Case Generation from Requirements: A Systematic Literature Review. *Computers, Materials Continua*, 67, 1819–1833. <https://www.techscience.com/cmc/v67n2/41326>
- Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., & Mian, A. (2024). A Comprehensive Overview of Large Language Models. *Computer Science > Computation and Language*, 10. <https://arxiv.org/abs/2307.06435>
- Nirpals, P. B., & Kale, K. V. (2011). A Brief Overview Of Software Testing Metrics. *International Journal on Computer Science and Engineering*, 3. https://www.researchgate.net/publication/50247283_A_Brief_Overview_Of_Software_Testing_Metrics
- obaidasajjad-SQA-Engineer. (2024). *OrangeHRM-Login-Testing* [abgerufen am 11.02.2025]. <https://github.com/obaidasajjad-SQA-Engineer/OrangeHRM-Login-Testing>
- OpenAI. (2025). *GPT-4 is OpenAI's most advanced system, producing safer and more useful responses* [abgerufen am 17.01.2025]. <https://openai.com/index/gpt-4/>
- OpenCart. (2025). *OpenCart* [abgerufen am 11.02.2025]. <https://www.opencart.com/>
- OrangeHRM. (2025). *OrangeHRM* [abgerufen am 11.02.2025]. <https://www.orangehrm.com/>
- Ouédraogo, W. C., Kaboré, K., Tian, H., Song, Y., Koyoncu, A., Klein, J., Lo, D., & Bissyandé, T. F. (2024). Large-scale, Independent and Comprehensive study of the power of LLMs for test case generation. *Computer Science > Software Engineering*, 1(1). <https://arxiv.org/abs/2407.00225>
- Pavankumar21github. (2022). *Manual-Testing-Project* [abgerufen am 11.02.2025]. <https://github.com/Pavankumar21github/Manual-Testing-Project>
- Salemi, A., & Zamani, H. (2024). Evaluating Retrieval Quality in Retrieval-Augmented Generation. *SIGIR '24: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2395–2400. <https://dl.acm.org/doi/10.1145/3626772.3657957>
- Schmid, A., & Sommer, M. (2024). *Retrieval Augmented Generation – RAG: Generative künstliche Intelligenz-Lösungen mit Unternehmensdaten anreichern* [abgerufen am 13.01.2025]. <https://aracom.de/retrieval-augmented-generation/>
- Siebert, J. (2024). *Halluzinationen von generativer KI und großen Sprachmodellen (LLMs)* [abgerufen am 22.11.2024]. <https://www.iese.fraunhofer.de/blog/halluzinationen-generative-ki-llm/>
- Statistisches Bundesamt. (2024). *Jedes fünfte Unternehmen nutzt künstliche Intelligenz* [abgerufen am 05.11.2024]. https://www.destatis.de/DE/Presse/Pressemitteilungen/2024/11/PD24_444_52911.html#:~:text=444%20vom%2025.,November%20

- 202024&text=WIESBADEN%20%E2%80%93%20Jedes%20f%C3%BCnfte%20Unternehmen%20(20, Einheiten%20mit%20mindestens%20zehn%20Besch%C3%A4ftigten.
- Streim, A., & Hecker, J. (2023). *Deutsche Wirtschaft drückt bei Künstlicher Intelligenz aufs Tempo* [abgerufen am 03.11.2024]. <https://www.bitkom.org/Presse/Presseinformation/Deutsche-Wirtschaft-drueckt-bei-Kuenstlicher-Intelligenz-aufs-Tempo>
- Tran, H. K. V., b[in] A[li], N., Unterkalmsteiner, M., Böstler, J., & Chatzipetrou, P. (2025). Deep learning and machine vision for food processing: A survey. *Software Quality Journal*, 33. <https://link.springer.com/article/10.1007/s11219-024-09698-w>
- Visure. (2024). *Was ist Anforderungsspezifikation: Definition, beste Tools und Techniken* [abgerufen am 15.01.2025]. <https://visuresolutions.com/de/blog/requirements-specification/>
- Wang, X., & Zhu, D. (2024). Validating LLM-Generated Programs with Metamorphic Prompt Testing. *Computer Science > Software Engineering*, 1. <https://arxiv.org/abs/2406.06864>
- Weingartz, R., & Suleymanov, N. (2024). *Wie man bessere Tests mit KI schreibt* [abgerufen am 03.11.2024]. <https://aqua-cloud.io/de/ai-to-write-tests/>
- Wu, S., Xiong, Y., Cui, Y., Wu, H., Chen, C., Yuan, Y., Huang, L., Liu, X., Kuo, T., Guan, N., & Xue, C. J. (2024). Retrieval-Augmented Generation for Natural Language Processing: A Survey. *Computer Science > Computation and Language*, 2. <https://arxiv.org/abs/2407.13193>
- Yuan, W., Neubig, G., & Liu, P. (2021). BARTSCORE: Evaluating Generated Text as Text Generation. *Computer Science > Computation and Language*, 2. <https://arxiv.org/abs/2106.11520>
- Zhu, L., Spachos, P., Pensini, E., & Plataniotis, K. (2021). Deep learning and machine vision for food processing: A survey. *Current Research in Food Science*, 4, 233–249. <https://www.sciencedirect.com/science/article/pii/S2665927121000228?via%3Dihub>

Eidesstattliche Erklärung

Hiermit erkläre ich eidesstattlich, dass die vorliegende Arbeit von mir selbstständig und ohne unerlaubte Hilfe angefertigt wurde, insbesondere, dass ich alle Stellen, die wörtlich oder annähernd wörtlich oder dem Gedanken nach aus Veröffentlichungen und unveröffentlichten Unterlagen und Gesprächen entnommen worden sind, als solche an den entsprechenden Stellen innerhalb der Arbeit durch Zitate kenntlich gemacht habe, wobei in den Zitaten jeweils der Umfang der entnommenen Originalzitate kenntlich gemacht wurde. Ich bin mir bewusst, dass eine falsche Versicherung rechtliche Folgen haben wird.

Ort, Datum

Unterschrift

Anhang

In diesem Anhang befinden sich die von der KI erzeugten Testfälle.